

Autour de la résolution d'équations différentielles

Professeurs référents : M. Couffignal et M. El Mahbouby

Institut Polytechnique des Sciences Avancées - - -
3TS1/2 - - -

2022-2023



CIARLO Emilio Luca
CLARET Nathan
FABRE Lucas

1	INTRODUCTION	2
2	Méthode de Verlet	3
2.0.1	Question 1)	3
2.0.2	Question 2)	5
2.0.3	Question 3) partie théorique du problème à 3 corps	6
2.0.4	Question 4) Partie théorique du problème à n corps	7
2.0.5	Question 5) application du problème à 3 et n corps	7
3	Variations autours des équations de diffusions	21
3.0.1	1) Résolution de l'équation de la chaleur 2D	21
3.0.2	Question 1)	21
3.0.3	Question 2)	22
3.0.4	Question 3)	24
3.0.5	Question 4) RKimage	25
3.0.6	2) Résolutions numériques de quelques variantes	28
3.0.7	Question 1) RKimagenormgradconstant	28
3.0.8	Question 2) RKimagenormgrad	29
3.0.9	Question 3) RKimagenormlaplace	30
3.0.10	Question 4)	32
3.0.11	Question 5)	32
3.0.12	3) Diffusion anisotrope	35
3.0.13	Question 1)	35
3.0.14	Question 2)	37
4	Conclusion	40

L'étude des équations différentielles est un domaine passionnant des mathématiques appliquées qui permet de modéliser des phénomènes naturels complexes tels que le mouvement des planètes et la diffusion de la chaleur. Les équations différentielles sont des équations qui décrivent comment une quantité varie en fonction du temps ou de l'espace. La résolution de ces équations peut donner des informations cruciales sur la dynamique de systèmes physiques, biologiques ou économiques.

Dans ce rapport, nous allons explorer deux exemples importants d'équations différentielles : l'équation de mouvement des planètes et l'équation de la chaleur en 2D. Nous verrons comment ces équations peuvent être résolues numériquement pour obtenir des prévisions précises sur le comportement des planètes dans le système solaire et la propagation de la chaleur dans des matériaux conducteurs.

La méthode de Verlet est une méthode numérique largement utilisée en physique et en ingénierie pour simuler le mouvement de particules dans des systèmes mécaniques. Elle a été développée pour la première fois dans les années 1960 par Loup Verlet pour résoudre des équations de mouvement pour les molécules dans la dynamique moléculaire. Depuis, elle a été étendue à une grande variété de problèmes, y compris la mécanique des fluides, la physique des particules, la mécanique quantique et la biologie.

La méthode de Verlet est une méthode de résolution d'équations différentielles qui est simple à implémenter et qui ne nécessite pas de calculer la dérivée première de la position ou de la vitesse d'un objet. Elle est basée sur des informations sur les positions et les accélérations actuelles et précédentes pour estimer la trajectoire future de l'objet. Cette méthode est donc particulièrement utile pour simuler des systèmes complexes à grande échelle.

2.0.1 Question 1)

1)a)

La méthode d'Euler explicite nous donne :

$$Z_{n+1} = Z_n + hf(t_n, y)$$

avec :

$$Z' = f(t, y)$$

On pose également :

$$z = \begin{pmatrix} X(t) \\ \dot{X}(t) \end{pmatrix}$$

$$\dot{z} = \begin{pmatrix} \dot{X}(t) \\ \frac{1}{m}F(X(t)) \end{pmatrix}$$

Et nous savons que :

$$\ddot{X}(t) = \frac{1}{m}F(X(t))$$

La méthode d'Euler explicite devient donc :

$$\begin{pmatrix} X(t) \\ \dot{X}(t) \end{pmatrix}_{n+1} = \begin{pmatrix} X(t) \\ \dot{X}(t) \end{pmatrix}_n + \frac{t_f - t_0}{N} \begin{pmatrix} \dot{X}(t) \\ \frac{1}{m}F(X(t)) \end{pmatrix}_n$$

En posant trivialement que :

$$X_n := X(t_n)$$

On obtient alors la méthode d'Euler explicite, pour notre problème, suivante :

$$\begin{pmatrix} X(t_{n+1}) \\ \dot{X}(t_{n+1}) \end{pmatrix} = \begin{pmatrix} X(t_n) \\ \dot{X}(t_n) \end{pmatrix} + \frac{t_f - t_0}{N} \begin{pmatrix} \dot{X}(t_n)t \\ \frac{1}{m}F(X(t_n)) \end{pmatrix}$$

1)b)

En un instant t l'énergie totale du système est donnée par :

$$\frac{m}{2} \|\dot{X}(t_n)\|^2 + \phi(X(t_n)) = E$$

On rappelle qu'on avait posé :

$$X_n := X(t_n)$$

On obtient alors l'expression de l'énergie totale suivante :

$$\Leftrightarrow E_n = \frac{m}{2} \|\dot{X}_n\|^2 + \phi(X_n)$$

On cherche maintenant à calculer l'expression de $E_{n+1} - E_n$:

$$E_{n+1} - E_n = \frac{m}{2} \|\dot{X}_{n+1}\|_2^2 + \phi(X_{n+1}) - \frac{m}{2} \|\dot{X}_n\|_2^2 - \phi(X_n)$$

$$E_{n+1} - E_n = \frac{m}{2} (\|\dot{X}_{n+1}\|_2^2 - \|\dot{X}_n\|_2^2) + \phi(X_{n+1}) - \phi(X_n)$$

$$E_{n+1} - E_n = \frac{m}{2} (\|\dot{X}_{n+1}\|_2^2 - \|\dot{X}_n\|_2^2) + h \left(\frac{\phi(X_{n+1}) - \phi(X_n)}{h} \right)$$

$$E_{n+1} - E_n = \frac{m}{2} (\|\dot{X}_{n+1}\|_2^2 - \|\dot{X}_n\|_2^2) - h \dot{X}_n^T F(X_n)$$

On sait qu'avec la question précédente :

$$\dot{X}_{n+1} = \dot{X}_n + \frac{h}{m} F(X_n)$$

Ainsi :

$$\Leftrightarrow \frac{m}{2} (\|\dot{X}_n + \frac{h}{m} F(X_n)\|_2^2 - \|\dot{X}_n\|_2^2) - h \dot{X}_n^T F(X_n)$$

$$\Leftrightarrow \frac{m}{2} ((\dot{X}_n + \frac{h}{m} F(X_n))(\dot{X}_n^T + \frac{h}{m} F(X_n)^T) - \|\dot{X}_n\|_2^2) - h \dot{X}_n^T F(X_n)$$

$$\Leftrightarrow \frac{m}{2} (\dot{X}_n^T \dot{X}_n + \frac{h}{m} \dot{X}_n^T F(X_n) + \frac{h}{m} F(X_n)^T \dot{X}_n + \frac{h^2}{m^2} F(X_n)^T F(X_n) - \|\dot{X}_n\|_2^2) - h \dot{X}_n^T F(X_n)$$

$$\begin{aligned} & \Leftrightarrow \cancel{\frac{m}{2} \dot{X}_n^T \dot{X}_n} + \frac{h}{2} \dot{X}_n^T F(X_n) + \frac{h}{2} F(X_n)^T \dot{X}_n + \frac{h^2}{2m} F(X_n)^T F(X_n) - \cancel{\frac{m}{2} \|\dot{X}_n\|_2^2} - h \dot{X}_n^T F(X_n) \\ & \Leftrightarrow -\frac{h}{2} \dot{X}_n^T F(X_n) + \frac{h}{2} F(X_n)^T \dot{X}_n + \frac{h^2}{2m} F(X_n)^T F(X_n) \end{aligned}$$

On sait que $\dot{X}_n$ et $F(X_n)$ sont des vecteurs, donc $\dot{X}_n^T F(X_n) = F(X_n)^T \dot{X}_n$.

Il nous reste donc :

$$E_{n+1} - E_n = \frac{h^2}{2m} F(X_n)^T F(X_n)$$

De plus, nous pouvons dire que : $\frac{h^2}{2m} F(X_n)^T F(X_n) > 0$ car $F(X_n)^T F(X_n) = f_1^2 + f_2^2 + \dots f_n^2 > 0$ avec f_n le terme ligne n du vecteur $F(X_n)$

Si la méthode d'Euler conservait l'énergie totale du système, cette dernière serait constante donc $E_{n+1} - E_n = 0$ Or ici, on a $E_{n+1} - E_n = \frac{h^2}{2m} F(X_n)^T F(X_n) > 0$ donc on a bien montré que la méthode d'Euler est non conservative.

2.0.2 Question 2)

2)a)

Grace au développement de Taylor du vecteur position à l'ordre 3 aux instant $t + h$ et $t - h$, on a :

$$\begin{aligned} X(t_n + h) &= X(t_n) + h \dot{X}(t_n) + \frac{h^2}{2} X^{(2)}(t_n) + \frac{h^3}{6} X^{(3)}(t_n) + h^3 \epsilon(h) \\ X(t_n - h) &= X(t_n) - h \dot{X}(t_n) + \frac{h^2}{2} X^{(2)}(t_n) - \frac{h^3}{6} X^{(3)}(t_n) + h^3 \epsilon(h) \end{aligned}$$

On considère h petit donc on peut dire que : $X(t_n + h) = X_n + 1$ et $X(t_n - h) = X_{n-1}$
En sommant les deux développements limités :

$$X_{n+1} + X_{n-1} = 2X_n + h^2 X_n^{(2)} + 2h^3 \epsilon(h)$$

$$X_{n+1} = 2X_n + h^2 X_n^{(2)} + 2h^3 \epsilon(h) - X_{n-1}$$

On sait que : $X^{(2)}(t) = \frac{1}{m} F(X(t)) = \frac{1}{m} F(X_n)$

On a donc :

$$X_{n+1} = 2X_n - X_{n-1} + \frac{h^2}{m} F(X_n)$$

s

2)b)

Notre schéma est d'ordre supérieur à 1 (ordre 2) donc il faut prendre une approximation de l'initialisation assez précise et donc qui est supérieur à l'ordre 1 afin d'éviter que l'erreur se diffuse dans l'algorithme de Verlet. On a trouvé à la question précédente : $X_{n+1} = 2X_n - X_{n-1} + \frac{h^2}{m} F(X_n)$
On approxime donc l'initialisation à l'ordre 2 grace à un développement de taylor à l'ordre 2 :

$$X(t_n - h) = X(t_n) - h\dot{X}(t_n) + \frac{h^2}{2}X^{(2)}(t_n) + h^2\epsilon(h)$$

Avec $n = 0$ (condition initiale) on a :

$$X(t_n - h) = X_{-1} \text{ et } X(t_n) = X_0 \text{ donc } \dot{X}(t_n) = \dot{X}_0 \text{ et } \ddot{X}(t_n) = \ddot{X}_0$$

Ainsi :

$$\Leftrightarrow X_{-1} = X_0 - h\dot{X}_0 + \frac{h^2}{2}X_0^{(2)}$$

De plus :

$$X_0^{(2)} = \frac{1}{m}F(X_0)$$

Donc :

$$X_{-1} = X_0 - h\dot{X}_0 + \frac{h^2}{m}F(X_0)$$

2.0.3 Question 3) partie théorique du problème à 3 corps

3)a)

Dans cette question nous allons définir les équations différentielles du mouvement pour le problème à 3 corps avec le Soleil, la Terre et Mars.

Grace au principe fondamental de la dynamique appliquée au Soleil on a :

$$\begin{aligned} m_S \frac{dv_S}{dt}(t) &= G \frac{m_S m_T}{||x_S - x_T||^3} |x_T - x_S| + G \frac{m_S m_M}{||x_S - x_M||^3} |x_M - x_S| \\ \Leftrightarrow \frac{dv_S}{dt}(t) &= G \frac{m_T}{||x_S - x_T||^3} |x_T - x_S| + G \frac{m_M}{||x_S - x_M||^3} |x_M - x_S| \end{aligned}$$

Grace au principe fondamental de la dynamique appliquée à la Terre on a :

$$\begin{aligned} m_T \frac{dv_T}{dt}(t) &= G \frac{m_T m_S}{||x_T - x_S||^3} |x_S - x_T| + G \frac{m_T m_M}{||x_T - x_M||^3} |x_M - x_T| \\ \Leftrightarrow \frac{dv_T}{dt}(t) &= G \frac{m_S}{||x_T - x_S||^3} |x_S - x_T| + G \frac{m_M}{||x_T - x_M||^3} |x_M - x_T| \end{aligned}$$

Grace au principe fondamental de la dynamique appliquée à Mars on a :

$$\begin{aligned} m_M \frac{dv_M}{dt}(t) &= G \frac{m_M m_S}{||x_M - x_S||^3} |x_S - x_M| + G \frac{m_M m_T}{||x_M - x_T||^3} |x_T - x_M| \\ \Leftrightarrow \frac{dv_M}{dt}(t) &= G \frac{m_S}{||x_M - x_S||^3} |x_S - x_M| + G \frac{m_T}{||x_M - x_T||^3} |x_T - x_M| \end{aligned}$$

3)b)

Par la suite nous allons définir le problème de cauchy, en effet nous posons un premier vecteur composé lui même des vecteurs positions et vitesses de chaque corps. Ensuite en dérivant Y on obtient ainsi les vecteurs vitesses et accélérations suivants :

$$Y = \begin{bmatrix} X_S \\ X_T \\ X_M \\ V_S \\ V_T \\ V_M \end{bmatrix} \quad Y' = \begin{bmatrix} V_S \\ V_T \\ V_M \\ A_S \\ A_T \\ A_M \end{bmatrix} = \begin{bmatrix} V_S \\ V_T \\ V_M \\ G \frac{m_T}{||x_S - x_T||^3} |x_T - x_S| + G \frac{m_M}{||x_S - x_M||^3} |x_M - x_S| \\ G \frac{m_S}{||x_T - x_S||^3} |x_S - x_T| + G \frac{m_M}{||x_T - x_M||^3} |x_M - x_T| \\ G \frac{m_S}{||x_M - x_S||^3} |x_S - x_M| + G \frac{m_T}{||x_M - x_T||^3} |x_T - x_M| \end{bmatrix}$$

2.0.4 Question 4) Partie théorique du problème à n corps

Dans cette deuxième application, nous allons tenter de généraliser la méthode de Verlet pour le problème à n corps. Pour ce qui est des conditions initiales, nous utilisons une base de données récupérée sur le site de l'IMCCE. Nous avons choisi de prendre les valeurs des planètes de l'ensemble du système solaire. La date choisie est celle du 27 Avril 2020 à minuit.

4)a) Équations différentielles du mouvement pour n corps

Dans cette première question, nous déterminons les équations différentielles du mouvement pour n corps. Avec une méthode similaire que pour les équations du problème à 3 corps. On commence bien évidemment avec le principe fondamental de la dynamique tel que :

$$\frac{dv_i}{dt}(t) = G \sum_{j=1}^n \frac{m_j}{||x_i - x_j||^3} |x_j - x_i|$$

$$\Leftrightarrow \begin{bmatrix} \frac{dv_1}{dt}(t) \\ \vdots \\ \frac{dv_n}{dt}(t) \end{bmatrix} = \begin{bmatrix} \sum_{j=1, j \neq 1}^n G \frac{m_j}{||x_i - x_j||^3} |x_j - x_i| \\ \vdots \\ \sum_{j=n, j \neq n}^n G \frac{m_j}{||x_i - x_j||^3} |x_j - x_i| \end{bmatrix}$$

4)b) Problème de Cauchy

Ici nous appliquons le problème de Cauchy au problème à n corps. De la même manière que pour le problème à 3 corps, nous définissons un vecteur Y qui regroupe l'ensemble des vecteurs positions et vitesses des corps allant de 1 à n.

Ensuite en dérivant Y on obtient la dérivée de chaque vecteur positions et vitesses pour obtenir les vecteurs vitesses et accélérations allant de 1 à n.

$$Y = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_N \\ V_1 \\ V_2 \\ \vdots \\ V_N \end{bmatrix} \Leftrightarrow Y' = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_N \\ A_1 \\ A_2 \\ \vdots \\ A_N \end{bmatrix} = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_N \\ \sum_{j=1, j \neq 1}^n G \frac{m_j}{||x_i - x_j||^3} |x_j - x_i| \\ \vdots \\ \sum_{j=n, j \neq n}^n G \frac{m_j}{||x_i - x_j||^3} |x_j - x_i| \end{bmatrix}$$

2.0.5 Question 5) application du problème à 3 et n corps

Afin de commencer notre code python nous devons importer plusieurs bibliothèques et définir les différents paramètres donnés dans les consignes. On obtient ainsi :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4 import matplotlib.pyplot as plt
5 from numba import njit
6 import time
7
8 LONG = 1*10**(11)
9 MASSE = 1*10**(24)
10 JOUR=24*3600
11 G = 6.67*10**(-11)/(LONG**3/MASSE/JOUR**2)
```

Nous allons ici programmer une fonction Python f (x_i, x_j, m_i, m_j) permettant de calculer la force exercée par un corps i sur un corps j. Nous allons pour cela nous servir de la loi de gravitation universelle qui dit que la force exercée par un corps A sur un corps B s'exprime de la manière suivante :

$$F_{A/B} = G \frac{m_A \cdot m_B}{d^2}$$

On a donc la fonction python suivante :

Programme de la fonction f :

```
1 def f(xi,xj,mi,mj):
2     d=np.linalg.norm(xi-xj)
3     return -G*((mi*mj)/(d**3))*(xj-xi)
```

Runge-Kutta 4

La fonction qui suit permet de calculer à l'aide du principe fondamental de la dynamique, l'accélération d'une planète i en fonction des forces d'attraction gravitationnelles que toutes les autres planètes appliquent à la planète i. Cette fonction renvoie le vecteur "F" qui est composé des vitesses des planètes (en x, y et z) dans sa première moitié, puis toutes les accélérations des planètes.

Contrairement à l'algorithme de Verlet, les vitesses sont indispensables à l'algorithme de Runge-Kutta pour actualiser les positions. C'est pourquoi nous avons créé une autre fonction "Force-verlet" que nous allons traiter juste après.

Programme de la fonction Force RK4 :

```
1 def Force_RK4(t,u,nbcorps,m):
2     F=np.zeros((nbcorps*6,1))
3     for i in range (nbcorps):
4         S=np.zeros((3,1))
5         for j in [x for x in range(nbcorps) if x != i]:
6             S+=f(u[(j*3):(j+1)*3].reshape(-1,1),u[(i*3):(i+1)*3].reshape(-1,1),
7                 m[j],m[i])
8         F[i*3:(i+1)*3,0]=u[nbcorps*3+i*3:(i+1)*3+nbcorps*3,0]
9         F[nbcorps*3+i*3:(i+1)*3+nbcorps*3,0]=((1/m[i])*S).reshape(-1)
10    return F
```

Ensuite on implémente une fonction Python utilisant la méthode de Runge-Kutta à l'ordre 4 pour résoudre le problème à n Corps. Cette fonction se sert de la fonction précédente pour calculer les vitesses et les accélérations des planètes à chaque itération afin de calculer une approximation numérique des équations différentielles associées aux problèmes à n corps : les positions des planètes.

Programme de la fonction RK4 :

```
1 def RK4(u0,h,t0,nbiter,nbcorps,m):
2     start = time.time()
3
4     t=t0
5     u=u0
6     U=np.zeros((3*nbcorps,nbiter))
7     for i in range(0,nbiter):
8         t=t0+h
9         k1=h*Force_RK4(t,u,nbcorps,m)
10        k2=h*Force_RK4(t+h/2,u+k1/2,nbcorps,m)
11        k3=h*Force_RK4(t+h/2,u+k2/2,nbcorps,m)
12        k4=h*Force_RK4(t+h,u+k3,nbcorps,m)
13        u=u+(1/6)*(k1 +2*k2 +2*k3 + k4)
14        for j in range (0,nbcorps):
15            U[j*3:(j+1)*3,i]=u[j*3:(j+1)*3,0]
16        print('it ration : {}'.format(i))
17
18    print(time.time() - start)
19    return U
```

Verlet

La fonction qui suit est similaire à la fonction "Force-RK4" à la seule différence que le vecteur "F" de celle-ci contient uniquement les accélérations des planètes. En effet, l'algorithme de Verlet n'a besoin que des accélérations actualisées pour calculer une approximation de la position des planètes.

Programme de la fonction Force verlet :

```
1 def Force_verlet(t,position,nbcorps,m):
2     F=np.zeros((nbcorps*3,1))
3     for i in range (nbcorps):
```

```

4      S=np.zeros((3,1))
5      for j in [x for x in range(nbcorps) if x != i]:
6          S+=f(position[(j*3):(j+1)*3].reshape(-1,1),position[(i*3):(i+1)*3].
              reshape(-1,1),m[j],m[i])
7      F[i*3:(i+1)*3,0]=((1/m[i])*S).reshape(-1)
8      return F

```

Ensuite on implémente une fonction Python utilisant la méthode de Verlet pour résoudre le problème à 3 Corps. Pour ce faire, nous suivons l'algorithme suivant :

$$\begin{cases} X_0, \dot{X}_0 \text{ données} \\ X_{-1} = X_0 - \dot{X}_0 h + 2mF(X_0) \\ X_{n+1} = 2X_n - X_{n-1} + mhF(X_n) \quad n \geq 0 \end{cases}$$

Programme de la fonction Verlet :

```

1 def verlet(position,vitesse,t0,nbiter,h,nbcorps,m):
2     start = time.time()
3     t=t0
4     Xn=position
5     Xpn=vitesse
6     F=Force_verlet(t,Xn,nbcorps,m)
7     X_1 = Xn - (h*Xpn) + ((h**2)/2) * F
8     V=np.zeros((3*nbcorps,nbiter))
9     for i in range(nbiter):
10        Xn1=2*Xn - X_1 + ((h**2)*F)
11        X_1=Xn
12        Xn=Xn1
13        F=Force_verlet(t,Xn,nbcorps,m)
14        V[:,i]=Xn1.reshape(-1)
15        print('it ration_{}_{}'.format(i))
16    print(time.time() - start)
17    return V

```

Trajectoire de chaque corps

Pour tracer les trajectoires des différents corps, nous avons implémenté deux fonctions :

La première fonction "plot3d" permet d'afficher les positions des planètes itération par itération sans tracer leur trajectoire.

Programme de la fonction plot3d :

```

1 def plot3d(u0, h, t0, nbiter, nbcorps, m, noms_corps, U):
2     plt.style.use('dark_background')
3     fig = plt.figure()
4     ax = fig.add_subplot(111, projection='3d')
5     couleurs = {
6         0: "red",
7         1: "green",
8         2: "blue",
9         3: "yellow",
10        4: "orange",
11        5: "purple",
12        6: "pink",
13        7: "gray",
14        8: "turquoise"}
15    noms_couleurs = {noms_corps[i]: couleurs[i] for i in range(nbcorps)}
16    for i in range(nbiter):
17        for j in range(nbcorps):
18            x, y, z = U[j*3,i], U[j*3+1,i], U[j*3+2,i]

```

```

19     nom = noms_corps[j]
20     couleur = noms_couleurs[nom]
21     ax.scatter3D(x, y, z, c=couleur)
22     ax.text(x, y, z, nom)
23     ax.set_xlabel('xUA')
24     ax.set_ylabel('yUA')
25     ax.set_zlabel('zUA')
26     ax.set_title('Simulation de mouvement de {} corps c lestes (it ration
    {})' .format(nbcors, i+1))
27     ax.set_xlim3d(-8, 8)
28     ax.set_ylim3d(-8, 8)
29     ax.set_zlim3d(-8, 8)
30     plt.pause(0.001)
31     plt.cla()

```

Nous utiliserons plutôt pour tracer les trajectoires la fonction "plot3d-2" qui trace les trajectoires au cours du temps ce qui est beaucoup plus visuel pour observer les ellipses complètes des planètes autour du soleil.

Programme de la fonction plot3d2 :

```

1  def plot3d_2(u0, h, t0, nbiter, nbcors, m, noms_corps, U, xyzmin, xyzmax):
2      fig = plt.figure()
3      ax = fig.add_subplot(111, projection='3d')
4      couleurs = {
5          0: "red",
6          1: "green",
7          2: "blue",
8          3: "yellow",
9          4: "orange",
10         5: "purple",
11         6: "pink",
12         7: "gray",
13         8: "turquoise"}
14     noms_couleurs = {noms_corps[i]: couleurs[i] for i in range(nbcors)}
15
16     for i in range(nbcors):
17         x = U[3*i, :]
18         y = U[3*i+1, :]
19         z = U[3*i+2, :]
20         ax.plot3D(x, y, z, color=couleurs[i])
21         ax.scatter3D(x[-1], y[-1], z[-1], color=couleurs[i], marker='o', s=50)
22         ax.text(x[-1], y[-1], z[-1], noms_corps[i], color=couleurs[i])
23
24     ax.set_xlabel('xUA')
25     ax.set_ylabel('yUA')
26     ax.set_zlabel('zUA')
27     ax.set_title('Simulation de mouvement de corps c lestes')
28     ax.set_xlim3d(xyzmin, xyzmax)
29     ax.set_ylim3d(xyzmin, xyzmax)
30     ax.set_zlim3d(xyzmin, xyzmax)
31     plt.show()

```

Tracé de normes

La fonction norme a pour objectif de calculer et de tracer la différence à chaque itération entre les positions des planètes données par Verlet et celles données par Runge-Kutta. Nous pourrions ainsi comparer la précision des deux algorithmes.

Programme de la fonction norme :

```

1  def norme(U,V):
2      norm = np.linalg.norm(U - V, axis=0)
3
4      plt.plot(norm)

```

```

5 plt.xlabel('Colonnes')
6 plt.ylabel('Norme')
7 plt.title('Norme entre les colonnes de U et V')
8 plt.show()
9 return np.linalg.norm(U,V)

```

Calculs d'énergies

Enfin, nous définissons les fonctions qui ont pour but de calculer les différentes énergies des planètes.

La fonction "ep" permet de calculer les énergies potentielles gravitationnelles de chaque planètes au fil du temps. On utilise pour cela la définition de l'énergie potentielle gravitationnelle d'un corps i qui est la suivante : $E_p(i) = -G \frac{m_i \cdot m_j}{d}$

On peut généraliser cette énergie au problème à n corps en sommant les énergies qu'appliquent toutes les planètes à la planète i :

$$\sum_{j=1}^{n_{\text{corps}}} -G \frac{m_i \cdot m_j}{d}$$

Avec m_i la masse du corps i , m_j la masse du corps j , G la constante de gravitation et d la distance entre les planètes i et j

Programme de la fonction qui calcule l'énergie potentielle de toutes les planètes :

```

1 def ep(m, planet, U, nbiter, X, titre):
2     nbcorps = len(planet)
3     ep_planete = np.zeros((nbcorps, nbiter))
4     for i in range(nbiter):
5         for j in range(nbcorps):
6             ep_planete_j = 0
7             for l in [x for x in range(nbcorps) if x != j]:
8                 d = np.linalg.norm(U[(j*3):(j+1)*3, i].reshape(-1,1) - U[(l*3):
9                     (l+1)*3, i].reshape(-1,1))
10                ep_planete_j += -(G*(m[j]*m[l])/d)
11            ep_planete[j, i] = ep_planete_j
12
13     if X==True:
14         plt.figure(figsize=(9,6))
15         for j in range(nbcorps):
16             plt.plot(range(nbiter), ep_planete[j, :], label=planet[j])
17         plt.xlabel('It rations')
18         plt.ylabel('Energie potentielle')
19         plt.title("Energie potentielle des plan tes au cours du temps avec l'
20             algorithme de {}".format(titre))
21         plt.legend()
22         plt.show()
23     return ep_planete

```

On définit une fonction "ep" qui calcule l'énergie potentielle de toutes les planètes. Pour ce faire, on utilise la très célèbre formule de l'énergie cinétique : $E_c = \frac{1}{2}mv^2$.

On utilisera la formule $v = \frac{d}{dt}$ pour calculer la vitesse des corps en prenant $d = position_t - position_{t-1}$. Il est clair qu'avec cette méthode, nous ne pouvons pas calculer l'énergie cinétique à l'instant $t = 0$ car il nous faudrait les positions à l'instant $t - 1$, or nous n'avons pas accès à ces dernières.

Programme de la fonction qui calcule l'énergie cinétique de toutes les planètes :

```

1 def ec(m, planet, U, nbiter, h, X, titre):
2     nbcorps = len(planet)
3     ec_planete = np.zeros((nbcorps, nbiter-1))
4     for i in range(1, nbiter):
5         for j in range(nbcorps):
6             v = (np.linalg.norm(U[(j*3):(j+1)*3, i] - U[(j*3):(j+1)*3, i-1]) /
7                 h)
8             ec_planete_j = (1/2) * m[j] * v**2
9             ec_planete[j, i-1] = ec_planete_j

```

```

10 if X==True:
11     plt.figure(figsize=(9,6))
12     for j in range(nbcorps):
13         plt.plot(range(nbiter-1), ec_planete[j, :], label=planet[j])
14     plt.xlabel('It rations')
15     plt.ylabel('Energie_cin tique')
16     plt.title("Energie_cin tique_des_plan tes_au_cours_du_temps_avec_l'
                algorithme_de_{}".format(titre))
17     plt.legend()
18     plt.show()
19     return ec_planete

```

On calcule à présent l'énergie totale des planètes en effectuant une simple somme : $E_{tot} = E_c + E_p$

Programme de la fonction qui calcule l'énergie totale de toutes les planètes :

```

1 def etot(m, planet, U, nbiter, h, X, titre):
2     nbcorps = len(planet)
3     e_cinetique=ec(m,planet,U,nbiter,h,X,titre)
4     e_potentielle=ep(m,planet,U,nbiter,X,titre)
5     e_tot=e_cinetique+e_potentielle[:,1:]
6
7     if X==True:
8         plt.figure(figsize=(9,6))
9         for j in range(nbcorps):
10             plt.plot(range(nbiter-1), e_tot[j, :], label=planet[j])
11         plt.xlabel('It rations')
12         plt.ylabel('Energie_totale_des_plan tes')
13         plt.title("Energie_totale_des_plan tes_(ec+ep)_au_cours_du_temps_avec_
                    l'algorithme_de_{}".format(titre))
14         plt.legend()
15         plt.show()
16     return e_tot,e_cinetique,e_potentielle

```

Enfin, nous définissons une fonction comparaison qui a pour but de sommer toutes les énergies totales de chaque planètes et à chaque itération pour avoir l'énergie totale du système. On effectue cela avec les données que nous renvoie Verlet et Runge-Kutta et on trace leur évolution sur le même graphique afin d'observer si les algorithmes conservent belle est bien l'énergie ou non.

Programme de la fonction qui somme les énergies totales de toutes les planètes :

```

1 def comparaison(etot_verlet, etot_RK4):
2     etot_systeme_verlet = np.sum(etot_verlet,axis=0)
3     etot_systeme_RK4 = np.sum(etot_RK4,axis=0)
4
5     plt.figure(figsize=(9,6))
6     plt.plot(etot_systeme_verlet, label='Verlet')
7     plt.plot(etot_systeme_RK4, label='RK4')
8     plt.xlabel('It rations')
9     plt.ylabel('Energie_totale')
10    plt.legend()
11    plt.title('Comparaison_de_la_m thode_de_Verlet_et_de_RK4')
12    plt.show()

```

Pour finir, nous avons codé une fonction qui sert à créer le vecteur qui contient les conditions initiales (position et vitesse) de chaque planète. Ces informations sont prises dans le fichier txt : "dataplanet" qui regroupe toutes les conditions initiales de chaque planète le 20 avril 2020 à 00 : 00
 Cette fonction renvoie également la masse des planètes que l'on souhaite tracer (plus d'informations dans le fichier main et lib de python).

Programme de la fonction qui crée le vecteur qui contient les conditions initiales (position et vitesse) de chaque planète :

```

1 def u0m(data,datalabel,planet,M):
2     position=np.zeros((len(planet)*3,1))
3     vitesse=np.zeros((len(planet)*3,1))
4     m=[]
5     for i in range(np.shape(datalabel)[0]): (nombre de ligne : 8)
6         for j in range(len(planet)):
7             if datalabel[i]==planet[j]:
8                 position[j*3:(j+1)*3]=data[i,0:3].reshape(-1,1)
9                 vitesse[j*3:(j+1)*3]=data[i,3:6].reshape(-1,1)
10    u0=np.concatenate((position,vitesse),axis=0)
11
12    for k in range(len(planet)):
13        m.append(M[planet[k]])
14    return u0,position,vitesse,m

```

Nous allons maintenant utiliser toutes les fonctions définies précédemment pour étudier le problème à 3 corps et celui à n corps et comparer la précision des algorithmes de Runge-Kutta à l'ordre 4 et celui de Verlet.

Problème à trois corps : Soleil, Terre, Mars

Commençons par tracer la trajectoire de trois corps : le soleil, la terre et Mars, grâce aux données initiales qui sont fournies dans le sujet. Nous choisirons un pas h de 0.1 et un nombre d'itérations de 15 000 pour les deux algorithmes pour pouvoir comparer leurs résultats.

Simulation des mouvements de 3 corps célestes avec la méthode de Runge-Kutta à l'ordre 4

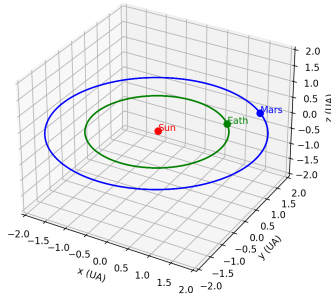


FIGURE 2.1 –

Simulation des mouvements de 3 corps célestes avec la méthode de Runge-Kutta à l'ordre 4

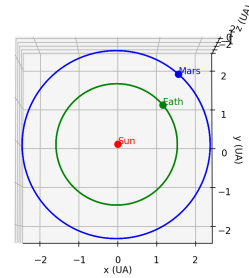


FIGURE 2.2 –

FIGURE 2.3 – Tracé des trajectoires avec l'algorithme de Runge-Kutta à l'ordre 4

Simulation des mouvements de 3 corps célestes avec la méthode de Verlet

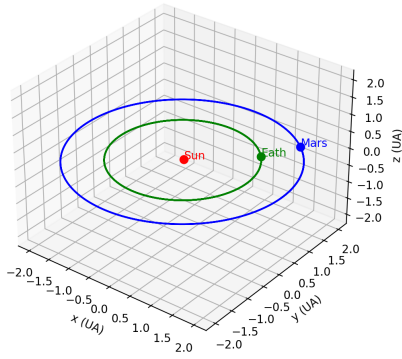


FIGURE 2.4 –

Simulation des mouvements de 3 corps célestes avec la méthode de Verlet

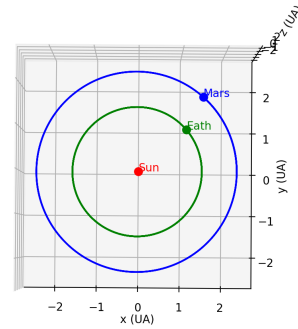
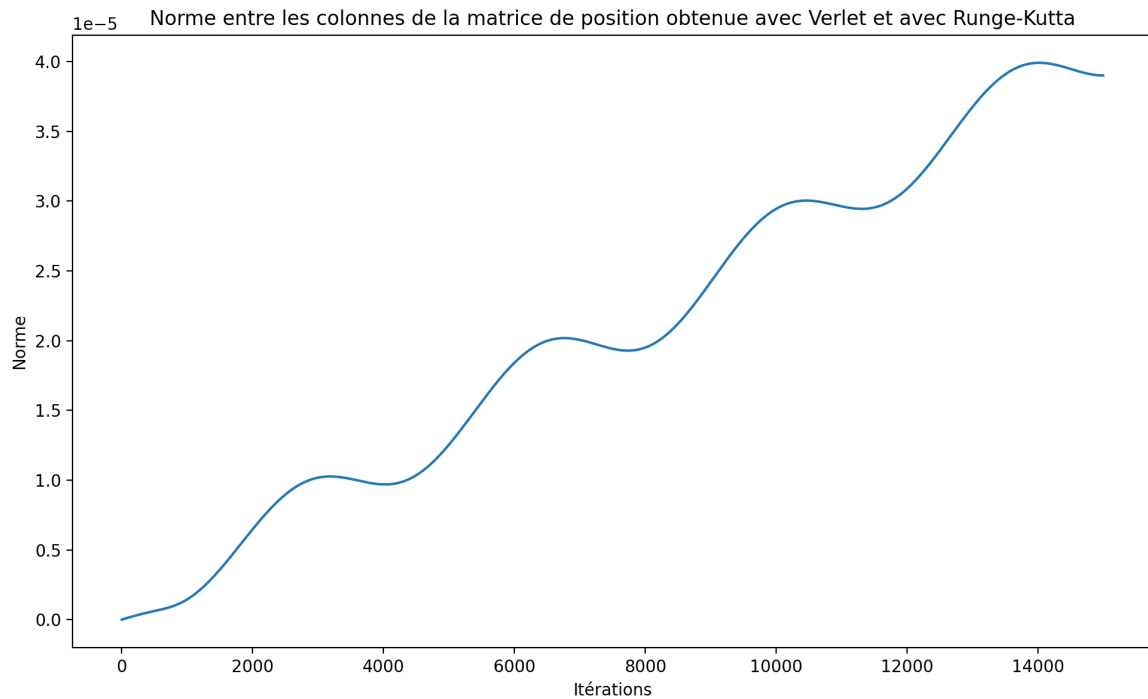


FIGURE 2.5 –

FIGURE 2.6 – Tracé des trajectoires avec l'algorithme de Verlet

On observe bien qu'avec un pas h de 0.1 et un nombre d'itérations égal à 15 000 les deux planètes ont une trajectoire circulaire autour du soleil qui ne se déplace pas et reste immobile au centre du graphique. On obtient quasiment les mêmes trajectoires en actualisant les positions des corps avec la méthode de Runge-Kutta à l'ordre 4, ou avec la méthode de Verlet. Pour comparer ces deux méthodes plus précisément, on utilisera la fonction norme :



Il est plus facilement possible de voir ici qu'il y a bien une différence entre les actualisations des positions que renvoie l'algorithme de Verlet et celui de Runge-Kutta à l'ordre 4. On note néanmoins que cette différence est très petite car elle atteint un maximum de $4 \times 10^{-5} \times LONG$ mètres ce qui est environ égal à 4×10^6 mètres soit 100 km ce qui est négligeable au vu de l'échelle du système solaire.

On peut également tracer les énergies cinétique et potentielle de chaque planètes avec la méthode de Runge-Kutta et la méthode de Verlet :

On peut observer que les énergies potentielles et cinétiques des planètes avec les deux méthodes sont pratiquement identiques. On remarque également que les résultats sont plutôt cohérents car le soleil qui est le corps qui bouge le moins il possède donc l'énergie cinétique la plus faible.

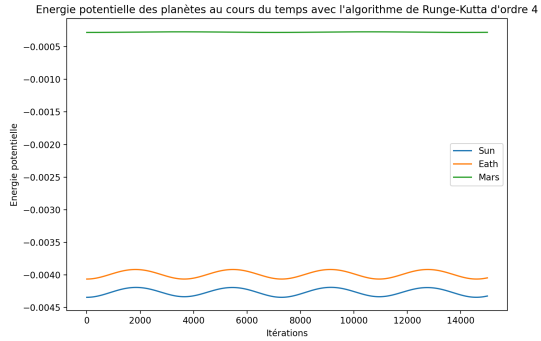


FIGURE 2.7 –

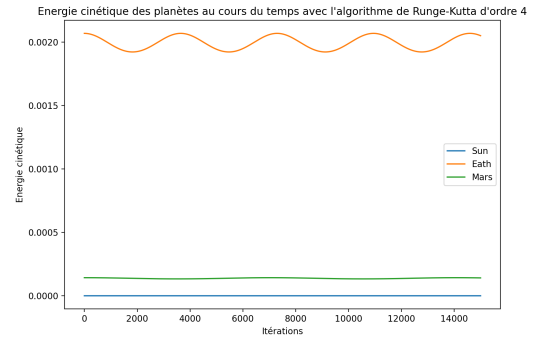


FIGURE 2.8 –

FIGURE 2.9 – énergie potentielle et cinétique pour Runge-Kutta à l'ordre 4

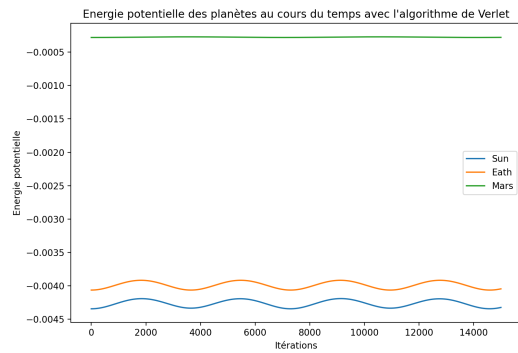


FIGURE 2.10 –

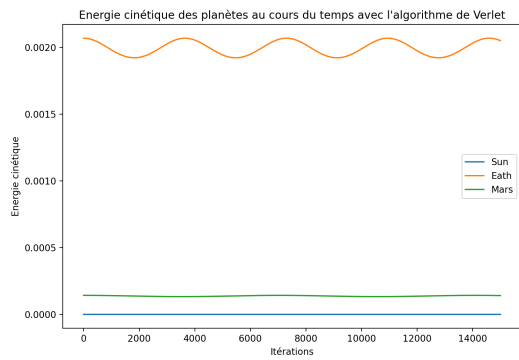


FIGURE 2.11 –

FIGURE 2.12 – énergie potentielle et cinétique pour Verlet

On peut maintenant tracer les énergies totales de chaque planètes :

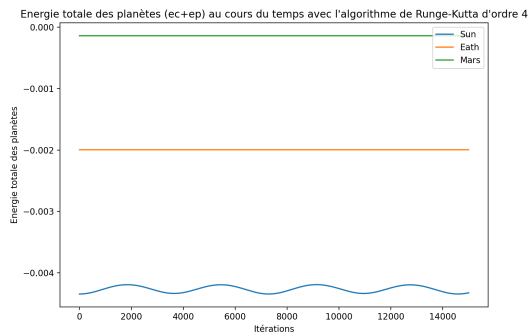


FIGURE 2.13 –

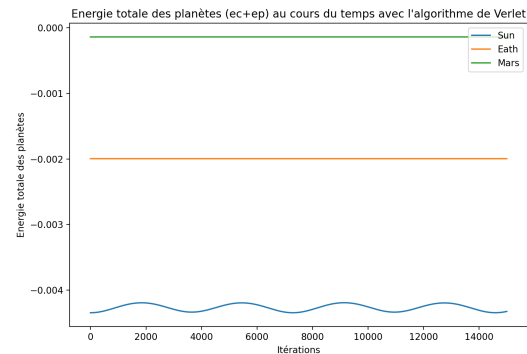
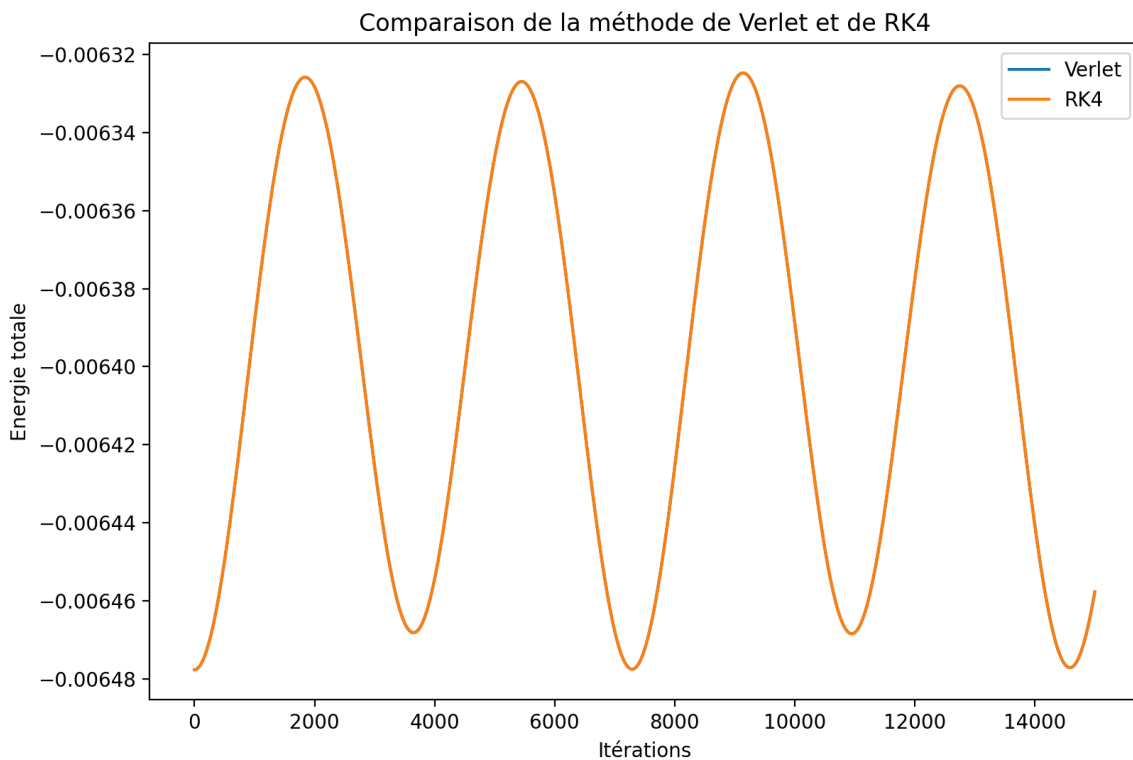


FIGURE 2.14 –

FIGURE 2.15 – énergie totale des planètes avec Runge-Kutta et Verlet

En sommant les énergies potentielles et cinétiques, on observe encore une fois que les deux méthodes sont quasiment identiques. On peut également noter que les énergies totales de la terre et de mars se conservent au cours du temps.



Enfin, on peut observer sur ce dernier graphique l'évolution de l'énergie totale du système à trois corps calculé à partir des positions données par Runge-Kutta et celles données par Verlet. Encore une fois, les deux courbes se superposent. On peut donc conclure qu'avec les conditions initiales de l'énoncé et avec un pas h de 0.1 et un nombre d'itérations de 15 000, les deux algorithmes ont des résultats très similaires.

Passons maintenant à l'étude de n corps céleste.

Nous prendrons pour cette étude les planètes du système solaire suivante : le soleil, Venus, la Terre, Mars, Jupiter, Saturn, Uranus et enfin Neptune pour un totale de 8 corps. Traçons dans un premier temps leur trajectoire avec un pas de 0.58 et 25 000 itérations :

Simulation des mouvements de 8 corps célestes avec la méthode de Runge-Kutta d'ordre 4

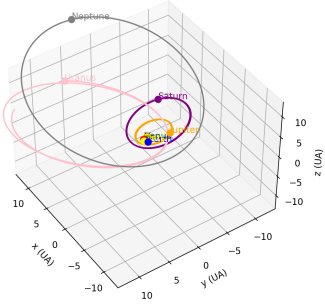


FIGURE 2.16 –

Simulation des mouvements de 8 corps célestes avec la méthode de Runge-Kutta d'ordre 4

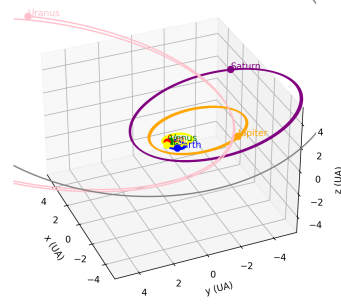


FIGURE 2.17 –

FIGURE 2.18 – Tracé des trajectoires avec l'algorithme de Runge-Kutta

Simulation des mouvements de 8 corps célestes avec la méthode de Verlet

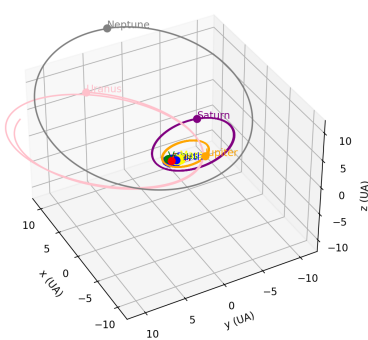


FIGURE 2.19 –

Simulation des mouvements de 8 corps célestes avec la méthode de Verlet

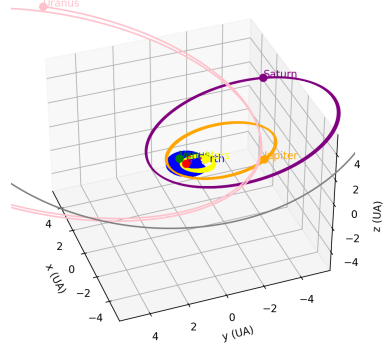
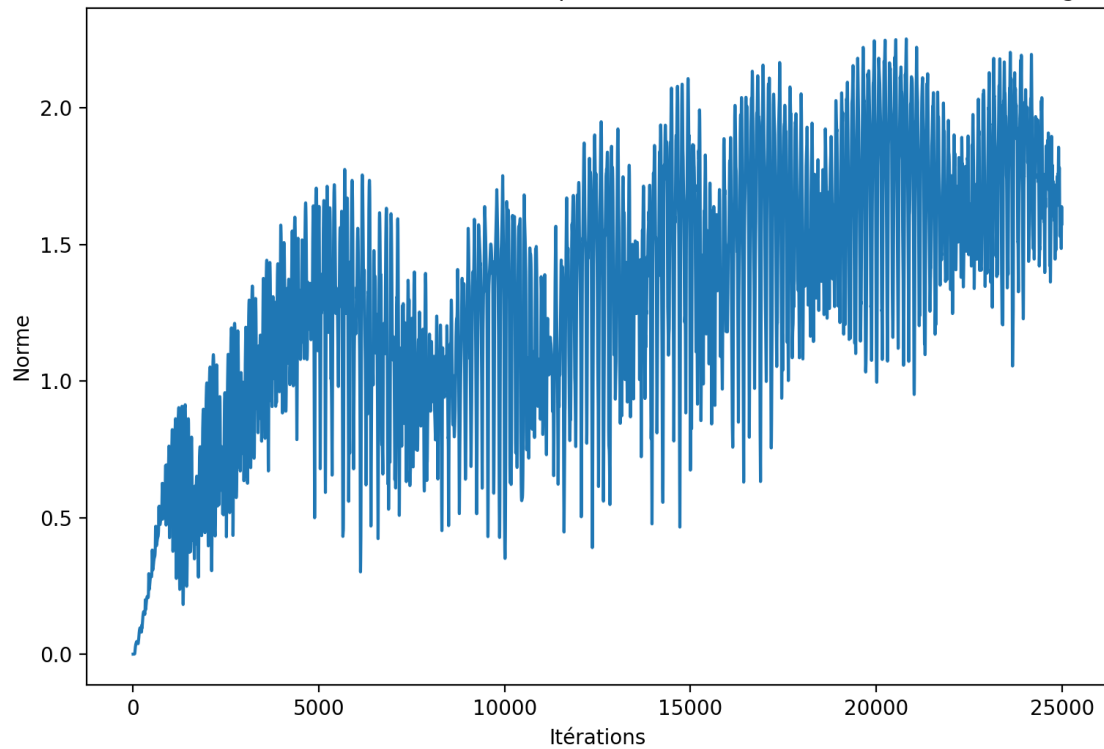


FIGURE 2.20 –

FIGURE 2.21 – Tracé des trajectoires avec l'algorithme de Verlet

De cette manière, on peut observer que toutes les planètes du système solaire tournent autour du soleil en suivant des trajectoires elliptiques et toute à peu près dans le même plan. Encore une fois, les deux algorithmes renvoient des trajectoires qui sont très semblables. Il faut zoomer au centre du système pour s'apercevoir que les trajectoires des planètes proches du soleil ne sont pas les mêmes pour les deux algorithmes (notamment celle de la Terre tracée en bleue).

Norme entre les colonnes de la matrice de position obtenue avec Verlet et avec Runge-Kutta



On peut observer ici que la norme entre les positions que renvoie Runge-Kutta et Verlet oscille énormément et ne fait qu'augmenter au fil des itérations. Cela montre bien les limites de l'algorithme de Runge-Kutta qui est moins précis pour ce genre de problème que l'algorithme de Verlet.

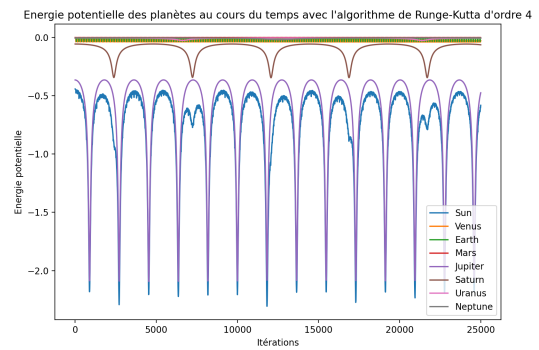
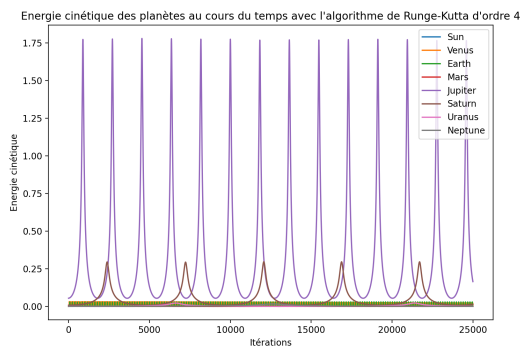


FIGURE 2.22 –

FIGURE 2.23 –

FIGURE 2.24 – énergie potentielle et cinétique pour Runge-Kutta

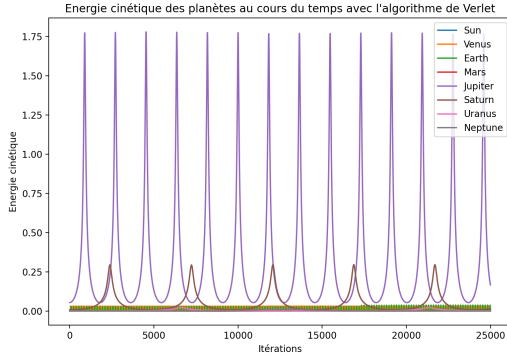


FIGURE 2.25 –

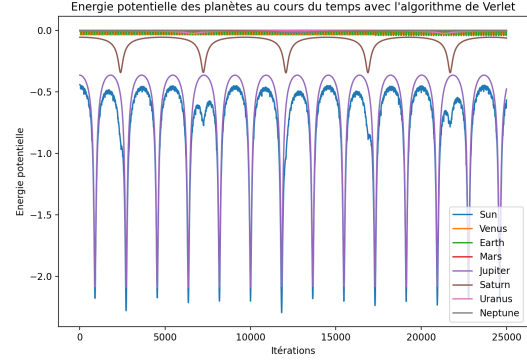


FIGURE 2.26 –

FIGURE 2.27 – énergie potentielle et cinétique pour Verlet

Quand aux énergies cinétique et potentielle de chaque planètes, on ne constate aucune différence entre Runge-Kutta et Verlet. On peut noter un phénomène intéressant de périodisation de l'énergie potentielle et de l'énergie cinétique du soleil, de Jupiter et de saturne. Ce phénomène peut s'expliquer par le fait que quand les planètes arrivent à leur périégées, leur vitesse augmente et leur distance avec le soleil est au minimum donc cela entraînant également une augmentation de l'énergie cinétique et potentielle (mais négativement pour ep).

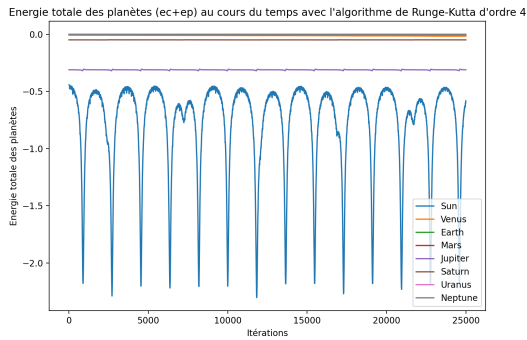


FIGURE 2.28 –

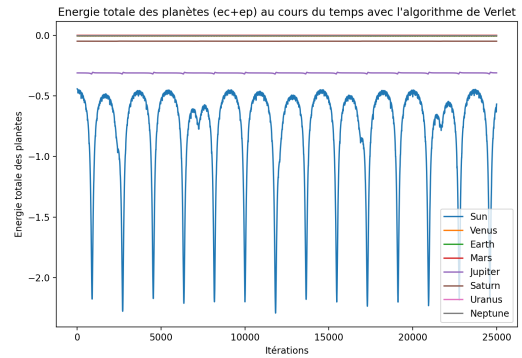
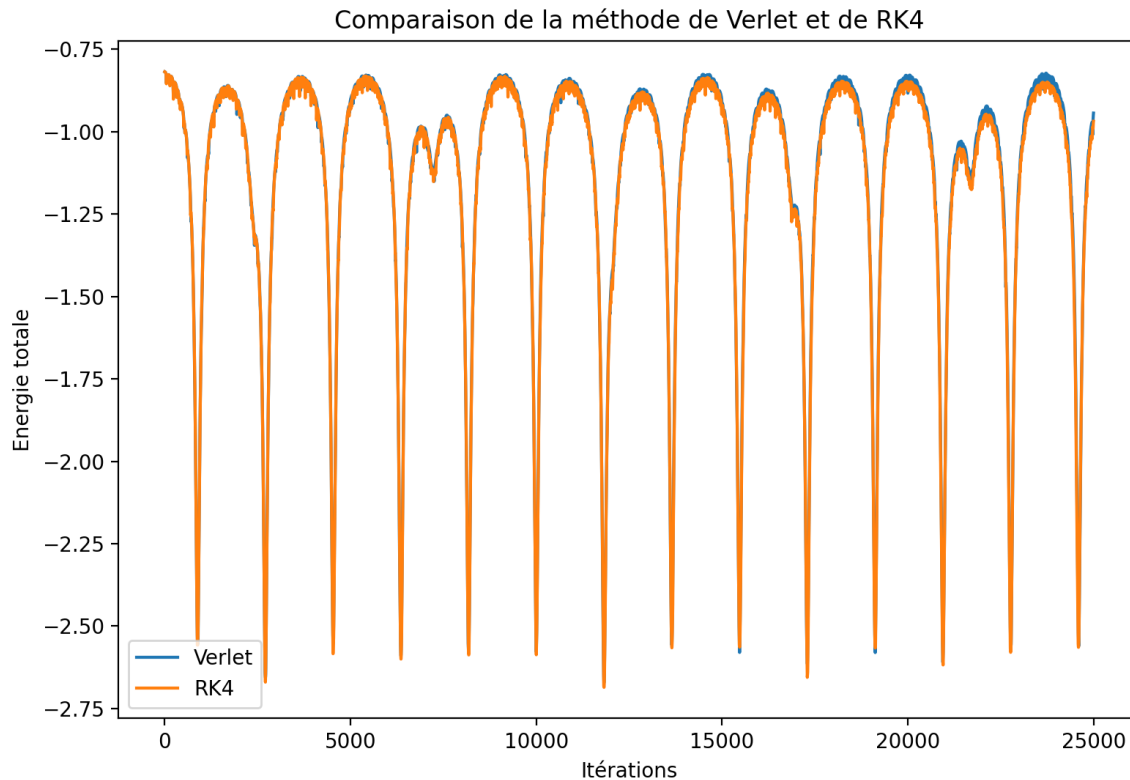


FIGURE 2.29 –

FIGURE 2.30 – énergie totale des planètes avec Runge-Kutta et avec Verlet

En sommant les énergies potentielles et cinétiques, on observe encore une fois que les deux méthodes sont quasiment identiques. On retrouve encore une fois cette périodicité surtout sur l'énergie totale du soleil.



Pour finir, on peut observer sur ce dernier graphique l'évolution de l'énergie totale du système à trois 8 corps calculé à partir des positions données par Runge-Kutta et celles données par Verlet. On peut observer que les courbes se superposent plutôt bien au début des itérations puis au fil du temps, les deux courbes se détache de plus en plus, marquant encore une fois la précision accrue de l'algorithme de Verlet.

En guise de conclusion

Cette étude nous a permis de montrer que l'algorithme de Verlet était très efficace dans ce genre de problème (problème à n corps célestes) car les forces qui s'appliquent sur les différents corps dans l'espace sont seulement celles des planètes entre elles (forces qu'on sait très facilement exprimer). C'est pourquoi ce puissant algorithme est souvent utilisé pour simuler des systèmes physiques en mécanique classique ou en chimie moléculaire car nous savons simplement exprimer les forces en jeu dans ces systèmes.

CHAPITRE 3

VARIATIONS AUTOUR DES ÉQUATIONS DE DIFFUSIONS

L'étude des équations de diffusion est essentielle dans le domaine de l'ingénierie, car ces équations décrivent la propagation de phénomènes physiques tels que la chaleur, la masse, ou encore la quantité de matière, dans un milieu donné. Lorsque le problème est complexe ou ne peut être résolu analytiquement, les méthodes numériques sont souvent utilisées pour résoudre ces équations. L'algorithme de Runge-Kutta 4 est l'une de ces méthodes numériques, et il est particulièrement efficace pour résoudre des équations de diffusion 2D. Il s'agit d'une méthode d'ordre élevé qui utilise un ensemble de formules pour estimer les valeurs de la solution à des points successifs dans le temps. Dans ce rapport d'ingénieur, nous allons présenter les équations de diffusion et leur importance dans le domaine de l'ingénierie, ainsi que l'algorithme de Runge-Kutta 4 et ses avantages en termes de résolution numérique. Nous allons également montrer comment l'algorithme de Runge-Kutta 4 peut être utilisé pour résoudre numériquement des variantes des équations de diffusion 2D, en utilisant des exemples concrets et des simulations informatiques. Enfin, nous allons discuter des résultats obtenus grâce à l'utilisation de l'algorithme de Runge-Kutta 4 pour résoudre ces variantes d'équations de diffusion 2D, en analysant leur précision et leur stabilité. Nous concluons en soulignant l'importance de l'algorithme de Runge-Kutta 4 pour la résolution numérique de problèmes de diffusion en ingénierie, ainsi que les perspectives pour de futures améliorations et applications.

3.0.1 1) Résolution de l'équation de la chaleur 2D

Dans cette section nous allons nous intéresser à la résolution numérique de l'équation de la chaleur suivante en posant $\gamma = 1$:

$$(P) = \begin{cases} \frac{\partial u}{\partial t}(t, x, y) = \Delta u(t, x, y) \\ u(0, x, y) = u_0(x, y) \\ \frac{\partial u}{\partial x}(t, x, 0) = \frac{\partial u}{\partial x}(t, x, 1) = 0 \\ \frac{\partial u}{\partial y}(t, 0, y) = \frac{\partial u}{\partial y}(t, 1, y) = 0 \end{cases}$$

De plus on se placera dans le carré unité : $\Omega := [0, 1]^2$. Fixons deux entiers $N \in \mathbb{N}^*$ et $M \in \mathbb{N}^*$ assez grand et notons $\Delta x = \frac{1}{M+1}$ et $\Delta y = \frac{1}{N+1}$. On a alors :

- un maillage du segment Ω par les points : $y_j := j\Delta y$, $x_i := i\Delta x$
- et on note alors : $u_{i,j}^t := u(t, x_i, y_j)$.

3.0.2 Question 1)

Dans cette première question, nous allons appliquer l'opérateur d'approximation des différences finies afin de montrer l'équation demandée pour les dérivées d'ordre 2.

On peut approximer la dérivée seconde en utilisant l'opérateur d'approximation des différences finies centrées. Ainsi, on a :

$$\Delta u(t, x_i, y_j) = \frac{\partial^2 u}{\partial x^2}(t, x_i, y_j) + \frac{\partial^2 u}{\partial y^2}(t, x_i, y_j)$$

On peut ainsi calculer chaque dérivée seconde séparément pour rendre le calcul moins lourd. Commençons par la dérivée seconde de x qui est :

$$\frac{\partial^2 u}{\partial x^2}(t, x_i, y_j) = \lim_{\Delta x \rightarrow 0} \frac{u(t, x_i + \Delta x, y_j) - 2u(t, x_i, y_j) + u(t, x_i - \Delta x, y_j)}{\Delta x^2}$$

On approxime maintenant la dérivée seconde en utilisant l'opérateur d'approximation des différences finies centrées tel que :

$$\frac{\partial^2 u}{\partial x^2}(t, x_i, y_j) \approx \frac{u(t, x_{i+1}, y_j) - 2u(t, x_i, y_j) + u(t, x_{i-1}, y_j)}{\Delta x^2}$$

Or on sait que : $u_{i,j}^t := u(t, x_i, y_j)$

On a alors :

$$\boxed{\frac{\partial^2 u}{\partial x^2}(t, x_i, y_j) \approx \frac{u_{i+1,j}^t - 2u_{i,j}^t + u_{i-1,j}^t}{\Delta x^2}}$$

Ensuite on continue avec la dérivée seconde de y qui est :

$$\frac{\partial^2 u}{\partial y^2}(t, x_i, y_j) = \lim_{\Delta y \rightarrow 0} \frac{u(t, x_i, y_j + \Delta y) - 2u(t, x_i, y_j) + u(t, x_i, y_j - \Delta y)}{\Delta y^2}$$

On approxime maintenant la dérivée seconde en utilisant l'opérateur d'approximation des différences finies centrées tel que :

$$\frac{\partial^2 u}{\partial y^2}(t, x_i, y_j) \approx \frac{u(t, x_i, y_{j+1}) - 2u(t, x_i, y_j) + u(t, x_i, y_{j-1})}{\Delta y^2}$$

Or on sait que : $u_{i,j}^t := u(t, x_i, y_j)$

On a alors :

$$\boxed{\frac{\partial^2 u}{\partial y^2}(t, x_i, y_j) \approx \frac{u_{i,j+1}^t - 2u_{i,j}^t + u_{i,j-1}^t}{\Delta y^2}}$$

Finalement quand on additionne les termes on obtient bien l'équation demandée telle que :

$$\boxed{\Delta u(t, x_i, y_j) \approx \left(\frac{u_{i+1,j}^t - 2u_{i,j}^t + u_{i-1,j}^t}{\Delta x^2} + \frac{u_{i,j+1}^t - 2u_{i,j}^t + u_{i,j-1}^t}{\Delta y^2} \right)}$$

3.0.3 Question 2)

Dans cette deuxième question nous devons déduire que pour tout $t \geq 0$, on a une certaine approximation de (P_D) . On part ainsi de l'équation de la chaleur suivante :

$$(P_D) = \begin{cases} \frac{\partial u}{\partial t}(t, x_i, y_j) = \left(\frac{u_{i+1,j}^t - 2u_{i,j}^t + u_{i-1,j}^t}{(\Delta x)^2} + \frac{u_{i,j+1}^t - 2u_{i,j}^t + u_{i,j-1}^t}{(\Delta y)^2} \right) \\ u(0, x_i, y_j) = u_0(x_i, y_j) \\ \frac{\partial u}{\partial x}(t, x_i, 0) = \frac{\partial u}{\partial x}(t, x_i, 1) = 0(1) \\ \frac{\partial u}{\partial y}(t, 0, y_j) = \frac{\partial u}{\partial y}(t, 1, y_j) = 0(2) \end{cases}$$

On a ainsi pour l'équation (1) :

$$\frac{\partial u}{\partial x}(t, x_i, 0) = \lim_{x \rightarrow 0} \frac{u(t, x_i + \Delta x, 0) - u(t, x_i, 0)}{\Delta x} = \frac{u(t, x_i + 1, 0) - u(t, x_i, 0)}{\Delta x}$$

Or on sait que : $u_i^t, j = u(t, x_i, y_j) \Leftrightarrow u_i^t, 0 = u(t, x_i, 0)$

Ainsi on obtient :

$$\boxed{\frac{\partial u}{\partial x}(t, x_i, 0) = \frac{u_{i+1,0}^t - u_{i,0}^t}{\Delta x}}$$

Passons maintenant de l'autre coté de l'équation, on sait que : $u_i^t, j = u(t, x_i, y_j), \Delta y = \frac{1}{N+1}$ et

$$y_j = j \Delta y \Leftrightarrow \Delta y = \frac{y_j}{j}$$

Ainsi :

$$\Leftrightarrow \frac{y_j}{j} = \frac{1}{N+1}$$

$$\Leftrightarrow y_j(N+1) = j$$

Or : $y_j = 1$

$$\Rightarrow j = N+1$$

Donc $u(t, x_{i+1}, 1) = u_{i+1, N+1}^t$

Ainsi on développe comme précédemment tel que :

$$\frac{\partial u}{\partial x}(t, x_i, 1) = \lim_{x \rightarrow 0} \frac{u(t, x_i + \Delta x, 1) - u(t, x_i, 1)}{\Delta x} = \frac{u(t, x_i + 1, 1) - u(t, x_i, 1)}{\Delta x}$$

$$\Leftrightarrow \boxed{\frac{\partial u}{\partial x}(t, x_i, 1) = \frac{u_{i+1, N+1}^t - u_{i, N+1}^t}{\Delta x}}$$

Finalement on obtient :

$$\frac{\partial u}{\partial x}(t, x_i, 0) = \frac{\partial u}{\partial x}(t, x_i, 1) = 0$$

$$\Leftrightarrow \boxed{\frac{u_{i+1,0}^t - u_{i,0}^t}{\Delta x} = \frac{u_{i+1, N+1}^t - u_{i, N+1}^t}{\Delta x} = 0}$$

On peut maintenant effectuer le même type de calcul en en dérivant par rapport à y pour l'équation (2), on a ainsi :

$$\frac{\partial u}{\partial y}(t, 0, y_j) = \lim_{y \rightarrow 0} \frac{u(t, 0, y_j + \Delta y) - u(t, 0, y_j)}{\Delta y} = \frac{u(t, 0, y_j + 1) - u(t, 0, y_j)}{\Delta y}$$

Or on sait que : $u_i^t, j = u(t, x_i, y_j) \Leftrightarrow u_0^t, j = u(t, 0, y_j)$

Ainsi on obtient :

$$\boxed{\frac{\partial u}{\partial y}(t, 0, y_j) = \frac{u_{0, j+1}^t - u_{0, j}^t}{\Delta y}}$$

Passons maintenant de l'autre coté de l'équation, on sait d'après l'énoncé que :

$$u_i^t, j = u(t, x_i, y_j), \Delta x = \frac{1}{M+1} \text{ et } x_i = i \Delta x \Leftrightarrow \Delta x = \frac{x_i}{i}$$

Ainsi :

$$\Leftrightarrow \frac{1}{M+1} = \frac{1}{M+1}$$

$$\Leftrightarrow x_i(M+1) = i$$

Or : $x_i = 1$

$$\Rightarrow i = M+1$$

Donc $u(t, 1, y_{j+1}) = u_{M+1,j+1}^t$

Ainsi on développe comme précédemment tel que :

$$\frac{\partial u}{\partial y}(t, 1, y_j) = \lim_{y \rightarrow 0} \frac{u(t, 1, y_j + \Delta y) - u(t, 1, y_j)}{\Delta y} = \frac{u(t, 1, y_j + 1) - u(t, 1, y_j)}{\Delta y}$$

$$\Leftrightarrow \boxed{\frac{\partial u}{\partial y}(t, 1, y_j) = \frac{u_{M+1,j+1}^t - u_{M+1,j}^t}{\Delta y}}$$

Finalement on obtient :

$$\frac{\partial u}{\partial y}(t, 0, y_j) = \frac{\partial u}{\partial y}(t, 1, y_j) = 0$$

$$\Leftrightarrow \boxed{\frac{u_{0,j+1}^t - u_{0,j}^t}{\Delta y} = \frac{u_{M+1,j+1}^t - u_{M+1,j}^t}{\Delta y} = 0}$$

On obtient ainsi finalement l'approximation de (P) demandée telle que :

$$(P_D) = \begin{cases} \frac{\partial u}{\partial t}(t, x_i, y_j) = \left(\frac{u_{i+1,j}^t - 2u_{i,j}^t + u_{i-1,j}^t}{(\Delta x)^2} + \frac{u_{i,j+1}^t - 2u_{i,j}^t + u_{i,j-1}^t}{(\Delta y)^2} \right) \\ u(0, x_i, y_j) = u_0(x_i, y_j) \\ \frac{u_{i+1,0}^t - u_{i,0}^t}{\Delta x} = \frac{u_{i+1,N+1}^t - u_{i,N+1}^t}{\Delta x} = 0 \\ \frac{u_{0,j+1}^t - u_{0,j}^t}{\Delta y} = \frac{u_{M+1,j+1}^t - u_{M+1,j}^t}{\Delta y} = 0 \end{cases}$$

3.0.4 Question 3)

3)a)

On sait que la première équation de (P_D) s'écrit matriciellement sous la forme :

$$\frac{\partial U^t}{\partial t} = D_1 \tilde{U}^t[:, 1 : N] + \tilde{U}^t[1 : M, :] D_2^T$$

où $\tilde{U}^t[:, 1 : N]$ est la sous-matrice de $\tilde{U}^t$ où l'on a conservé les colonnes de 1 à N . De même pour les lignes pour $\tilde{U}^t[1 : M, :]$. En utilisant l'équation au dessus, on peut associer les termes des matrices. On en déduit donc D_1 et D_2 :

$$D_1 = \frac{1}{(\Delta x)^2} \begin{bmatrix} 1 & -2 & 1 & \dots & 0 & 0 & 0 \\ 0 & 1 & -2 & \dots & 0 & 0 & 0 \\ 0 & 0 & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 1 & -2 & 1 \end{bmatrix}_{[M, M+2]}$$

$$D_2 = \frac{1}{(\Delta y)^2} \begin{bmatrix} 1 & -2 & 1 & \dots & 0 & 0 & 0 \\ 0 & 1 & -2 & \dots & 0 & 0 & 0 \\ 0 & 0 & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 1 & -2 & 1 \end{bmatrix}_{[N, N+2]}$$

On fixe $\frac{1}{(\Delta x)^2} = 1$ et $\frac{1}{(\Delta y)^2} = 1$
En

3)b)

On sait que le problème de Cauchy s'écrit de la forme suivante :

$$(\mathbb{P}) : \begin{cases} y(t_0) = y_0 \\ y'(t) = f(t, y(t)) \end{cases}$$

On peut en déduire $u'(t)$ et $u(t_0)$ des équations précédentes :

$$u'(t) = \frac{\partial U^t}{\partial t} = D_1 \tilde{U}^t[:, 1 : N] + \tilde{U}^t[1 : M, :] D_2^T$$

$$u(t_0) = u_0$$

On obtient donc le problème de Cauchy associé à (P_D) :

$$(\mathbb{P}_D) : \begin{cases} u(t_0, x_i, y_j) = u_0(x_i, y_j) \\ \frac{dU^t}{dt} = D_1 \tilde{U}^t[:, 1 : N] + \tilde{U}^t[1 : M, :] D_2^T \end{cases}$$

3)c)

On va écrire maintenant l'algorithme de Range Kutta associé à ce problème de Cauchy :

$$t_{k+1} = t_k + h$$

$$k_1 = hf(t_k, y_k) = \left(D_1 \tilde{U}_k^t[:, 1 : N] + \tilde{U}_k^t[1 : M, :] D_2^T \right) \frac{d^2 U_k^t}{dt^2}$$

$$k_2 = hf\left(t_k + \frac{h}{2}, y_k + \frac{k_1}{2}\right)$$

$$k_3 = hf\left(t_k + \frac{h}{2}, y_k + \frac{k_2}{2}\right)$$

$$k_4 = hf(t_k + h, y_k + k_3)$$

$$y_{k+1} = y_k + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Cet algorithme nous aidera dans la résolution des équations différentielles.

3.0.5 Question 4) RKimage

4)a) Complétion(U)

Dans cette question, nous allons rédiger une fonction Python complétion(U) qui prend en entrée une matrice image et qui renvoie la matrice $\tilde{U}$. Le but de cette fonction est de compléter la matrice U rentrée en doublant l'ensemble de ses bords. Dans un premier temps, nous avons utilisé la matrice U que l'on entourait de zéros mais nous nous sommes rapidement rendu compte que cela ne pouvait pas fonctionner.

On obtient ainsi le code suivant :

Programme de la fonction complétion(U) :

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 import cv2
4
5 def completion(U):
6     m,n= np.shape(U)
7     Ut = np.zeros((m+2, n+2))
8     Ut[1:-1, 1:-1] = U
9     Ut[0, 1:-1] = U[0, :]
10    Ut[-1, 1:-1] = U[-1, :]
11    Ut[1:-1, 0] = U[:, 0]
12    Ut[1:-1, -1] = U[:, -1]
13    Ut[0, 0] = U[0, 0]
```

```

14     Ut[-1, -1] = U[-1, -1]
15     Ut[0, -1] = U[0, -1]
16     Ut[-1, 0] = U[-1, 0]
17     return Ut

```

4)b) Dmat(n)

Par la suite, nous définissons une fonction Dmat(n) qui permet de calculer une sous-matrice de la matrice D utilisée dans f :

Programme de la fonction Dmat(n) :

```

1 def Dmat(n):
2     A=np.eye(n+2)
3     B=-2*np.eye(n+2, k=1)
4     C=np.eye(n+2,k=2)
5     D=A+B+C
6     return D[:n,:n+2]

```

4)c) RKImage(f,U0,t0,h,nbiter)

Dans un premier temps nous devons définir une fonction f(U) que nous implémenterons par la suite dans la fonction RKImage. Cette fonction f est tout simplement la fonction qui permet de calculer l'équation de la chaleur à partir de l'équation donnée et des fonctions complétion(U) et Dmat(n). On obtient ainsi le code suivant :

Programme de la fonction f(U) :

```

1 def f(U):
2     m,n=np.shape(U)
3     D1=Dmat(m)
4     D2=Dmat(n)
5     U_1=complétion(U)[:,:n+1]
6     U_2=complétion(U)[1:m+1,:]
7     return D1@U_1+U_2@D2.T

```

On peut maintenant définir la fonction RKImage qui calcule l'image U après nbiter itérations de l'équation de la chaleur avec la méthode de Runge-Kutta d'ordre 4 à partir de la fonction f précédente telle que :

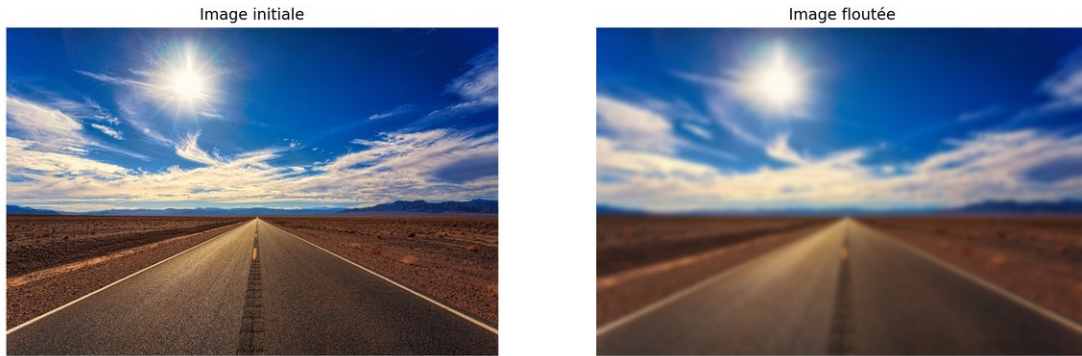
Programme de la fonction RKImage :

```

1 def RKImage(f,U0,t0,h,nbiter) :
2     U = U0.astype(np.float64)
3     m,n,c=np.shape(U)
4     L=np.zeros((m,n,c))
5     for j in range(c):
6         A=U[:, :, j]
7         for i in range(nbiter):
8             k1=h*f(A)
9             k2=h*f(A+k1/2)
10            k3=h*f(A+k2/2)
11            k4=h*f(A+k3)
12            A=A+(1/6)*(k1+2*k2+2*k3+k4)
13            print('iteration {}: {}'.format(i), A)
14
15        L[:, :, j]=A
16    return np.uint8(L)

```

On obtient ainsi la comparaison d'image suivante pour la photo de la route :



On voit à gauche l'image originale, et à droite l'image floutée. On peut voir que le gradient a été très efficace, puisque le pas était de 0.1, avec 50 itérations.

3.0.6 2) Résolutions numériques de quelques variantes

3.0.7 Question 1) RKimagenormgradconstant

Dans cette première question nous implémentons sur python le problème P suivant :

$$(P) = \begin{cases} \frac{\partial u}{\partial t}(t, x, y) = \|\nabla u_0(x, y)\|_2 \\ u(0, x, y) = u_0(x, y) \\ \frac{\partial u}{\partial x}(t, x, 0) = \frac{\partial u}{\partial x}(t, x, 1) = 0 \\ \frac{\partial u}{\partial y}(t, 0, y) = \frac{\partial u}{\partial y}(t, 1, y) = 0 \end{cases}$$

(3.1)

Tout d'abord, nous redéfinissons une fonction Dmat1(n) qui permet de calculer une sous-matrice de la matrice D utilisée dans fgrad :

Programme de la fonction Dmat1(n) :

```
1 def Dmat1(n):
2     A=-0.5*np.eye(n+2)
3     C=0.5*np.eye(n+2,k=2)
4     D=A+C
5     return D[:n,:n+2]
```

Ensuite nous devons définir une fonction fgrad(U) que nous implémenterons par la suite dans la fonction RKimage. Cette fonction f est tout simplement la fonction qui permet de calculer la norme du gradient de f(U) à partir de l'équation donnée et des fonctions complétion(U) et Dmat1(n). On obtient ainsi le code suivant :

Programme de la fonction f(U) :

```
1 def f_grad(U):
2     m, n = np.shape(U)
3     return np.sqrt((Dmat1(m)@complétion(U)[: ,1:n+1])**2 + (complétion(U)[1:m
4         +1, :]@Dmat1(n).T)**2)
```

On peut enfin définir la fonction RKimagenormgradconstant qui calcule l'image U après nbiter itérations de l'équation de la chaleur avec la méthode de Runge-Kutta d'ordre 4 à partir de la fonction f précédente telle que :

Programme de la fonction RKimagenormgradconstant :

```
1 def RKimage_normgradconstant(f, U0, t0, h, nbiter):
2     U = U0.astype(np.float64)
3     m,n,c=np.shape(U)
4     L=np.zeros((m,n,c))
5     for j in range(c):
6         A=U[:, :, j]
7         k1 = h * f_grad(A)
8         k2 = h * f_grad( A+ k1 / 2)
9         k3 = h * f_grad( A + k2 / 2)
10        k4 = h * f_grad( A + k3)
11        for i in range(nbiter):
12            A = A + (k1 + 2 * k2 + 2 * k3 + k4) / 6
13            print('iteration_{}_{}'.format(i))
14            L[:, :, j]=A
15    return L
```

Pour l'exemple donné dans le sujet avec la photo de la fraise on obtient en sortie la comparaison suivante :

On constate que la diffusion du gradient de l'image originale accentue les contours de l'image ainsi que sa brillance.

On obtient ainsi la comparaison d'image suivante pour la photo de la fraise :



Pour obtenir cette image nous avons appliqué à la fonction un pas de 0.4 et 10 itérations.

3.0.8 Question 2) RKimagenormgrad

Par la suite nous implémentons sur python le problème P suivant :

$$(P) = \begin{cases} \frac{\partial u}{\partial t}(t, x, y) = \|\nabla u(t, x, y)\|_2 \\ u(0, x, y) = u_0(x, y) \\ \frac{\partial u}{\partial x}(t, x, 0) = \frac{\partial u}{\partial x}(t, x, 1) = 0 \\ \frac{\partial u}{\partial y}(t, 0, y) = \frac{\partial u}{\partial y}(t, 1, y) = 0 \end{cases}$$

(3.2)

Tout d'abord, nous redéfinissons une fonction Dmat1(n) qui permet de calculer une sous-matrice de la matrice D utilisée dans fgrad :

Programme de la fonction Dmat1(n) :

```
1 def Dmat1(n):
2     A=-0.5*np.eye(n+2)
3     C=0.5*np.eye(n+2,k=2)
4     D=A+C
5     return D[:n,:n+2]
```

Ensuite nous devons définir une fonction fgrad(U) que nous implémenterons par la suite dans la fonction RKImage. Cette fonction f est tout simplement la fonction qui permet de calculer la norme du gradient de f(U) à partir de l'équation donnée et des fonctions complétion(U) et Dmat1(n). On obtient ainsi le code suivant :

Programme de la fonction f(U) :

```
1 def f_grad(U):
2     m, n = np.shape(U)
3     return np.sqrt((Dmat1(m)@completion(U)[: ,1:n+1])**2 + (completion(U)[1:m
4         +1, :]@Dmat1(n).T)**2)
```

On peut enfin définir la fonction RKimagenormgrad qui calcule l'image U après nbiter itérations de l'équation de la chaleur avec la méthode de Runge-Kutta d'ordre 4 à partir de la fonction f précédente telle que :

Programme de la fonction RKimagenormgrad :

```

1 def RKimage_normgrad(f, U0, t0, h, nbiter):
2     U = U0.astype(np.float64)
3     m,n,c=np.shape(U)
4     L=np.zeros((m,n,c))
5     for j in range (c):
6         A=U[:, :, j]
7         for i in range(nbiter):
8             k1 = h * f_grad(A)
9             k2 = h * f_grad( A+ k1 / 2)
10            k3 = h * f_grad( A + k2 / 2)
11            k4 = h * f_grad( A + k3)
12            A = A + (k1 + 2 * k2 + 2 * k3 + k4) / 6
13            print('iteration {}:{}'.format(i))
14        L[:, :, j]=A
15    return L

```

Afin de comparer avec la méthode précédente, nous pouvons également utiliser la photo de la fraise, on obtient en sortie la comparaison suivante :



Pour la fonction du gradient nous avons appliqué un pas de 0.2 avec 10 itérations. On peut constater que la norme du gradient par rapport à la norme du gradient constant est bien plus performant. En effet, le pas a été divisé par deux pour obtenir pratiquement la même image. Ici aussi il est possible voir que l'image modifié a les bords plus accentués, mais aussi qu'elle est plus éclairée comme la précédente.

3.0.9 Question 3) RKimagenormlaplace

Dans cette question, nous implémentons sur python le problème P suivant :

$$(P) = \begin{cases} \frac{\partial u}{\partial t}(t, x, y) = \sqrt{\left(\frac{\partial^2 u}{\partial x^2}(t, x, y)\right)^2 + \left(\frac{\partial^2 u}{\partial y^2}(t, x, y)\right)^2} \\ u(0, x, y) = u_0(x, y) \\ \frac{\partial u}{\partial x}(t, x, 0) = \frac{\partial u}{\partial x}(t, x, 1) = 0 \\ \frac{\partial u}{\partial y}(t, 0, y) = \frac{\partial u}{\partial y}(t, 1, y) = 0 \end{cases}$$

(3.3)

Tout d'abord, nous redéfinissons une fonction Dmat(n) qui permet de calculer une sous-matrice de la matrice D utilisée dans fgrad :

Programme de la fonction Dmat(n) :

```

1 def Dmat(n):

```

```

2  A=np.eye(n+2)
3  B=-2*np.eye(n+2, k=1)
4  C=np.eye(n+2,k=2)
5  D=A+B+C
6  return D[:n,:n+2]

```

Ensuite nous devons définir une fonction `fnormlaplace(U)` que nous implémenterons par la suite dans la fonction `RKImage`. Cette fonction `f` est tout simplement la fonction qui permet de calculer la norme de la laplacienne de $f(U)$ à partir de l'équation donnée et des fonctions `completion(U)` et `Dmat(n)`. On obtient ainsi le code suivant :

Programme de la fonction `fnormlaplace(U)` :

```

1  def fnormlaplace(U):
2      m, n = np.shape(U)
3      return np.sqrt((Dmat(m)@completion(U)[: ,1:n+1])**2 + (completion(U)[1:m
        +1, :]@Dmat(n).T)**2)

```

(lstlisting

On peut enfin définir la fonction `RKimagenormlaplace` qui calcule l'image U après `nbiter` itérations de l'équation de la chaleur avec la méthode de Runge-Kutta d'ordre 4 à partir de la fonction `f` précédente telle que :

Programme de la fonction `RKimagenormlaplace` :

```

1  def RKImage_normlaplace(U0, t0, h, nbiter):
2      U = U0.astype(np.float64)
3      m,n,c=np.shape(U)
4      L=np.zeros((m,n,c))
5      for j in range(c):
6          A=U[:, :,j]
7          for i in range(nbiter):
8              k1 = h * f_normlaplace(A)
9              k2 = h * f_normlaplace( A+ k1 / 2)
10             k3 = h * f_normlaplace( A+ k2 / 2)
11             k4 = h * f_normlaplace( A+ k3)
12             A = A + (k1 + 2 * k2 + 2 * k3 + k4) / 6
13             print('iteration_{}_{}'.format(i))
14             L[:, :,j]=A
15     return cv2.convertScaleAbs(L)

```

Afin de comparer avec la méthode précédente, nous pouvons également utiliser la photo de la fraise, on obtient en sortie la comparaison suivante :



Cette fois-ci, nous avons utilisé la méthode du laplacien. En effet nous avons appliqué la fonction `normlaplace`. On mesure ici l'accélération avec laquelle l'image est modifié (laplacien = dérivée seconde), alors que pour les images d'avant nous avons travaillé avec la vitesse (gradient = dérivée première).

Nous avons appliqué un pas de 0.08, et 10 itérations. On constate que le pas a été encore divisé par deux par rapport à la méthode précédente, pour obtenir encore une fois la même image. On peut donc en conclure que celle-ci est la méthode la plus "rapide", même si ce que l'on mesure avec le laplacien n'est pas la même chose de ce que l'on a mesuré auparavant.

3.0.10 Question 4)

Nous n'avons malheureusement pas réussi à tester d'autres fonctions à discretiser à droite. Cependant nous avons quand même essayé de tester l'algorithme avec d'autres équations, plus précisément l'équation d'onde. En faisant cela, même si nous n'avons pas réussi à l'appliquer à notre algorithme, nous avons pu en tirer des informations importantes. En essayant de mieux comprendre la démarche de notre fonction $f(U)$, nous avons compris que son but était similaire à celui de la fonction Canny dans la bibliothèque cv2, dont on en parlera plus tard. Ceci nous a permis aussi de comprendre ce que faisait la fonction `cv2.convertScalesAbs`. Dans les codes Python, il est possible de voir, surtout pour la partie précédente, cette fonction. Voici une représentation graphique de ce que fait cette fonction :

On peut voir à droite l'image sans la fonction `convertScalesAbs`, et à gauche l'image après utilisation

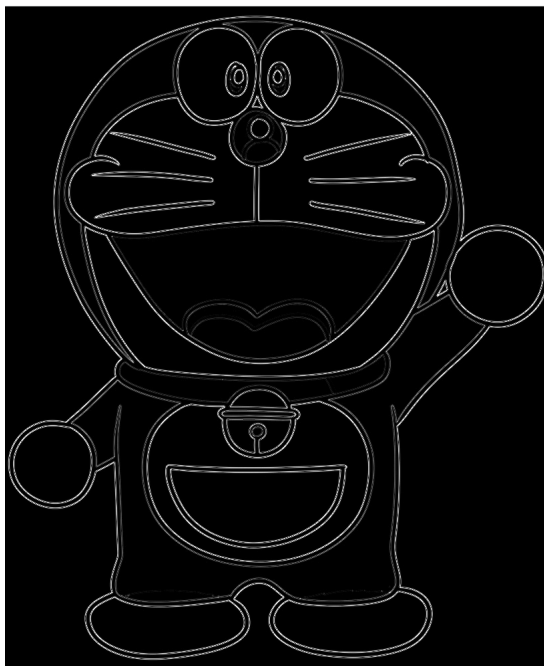


FIGURE 3.1 –

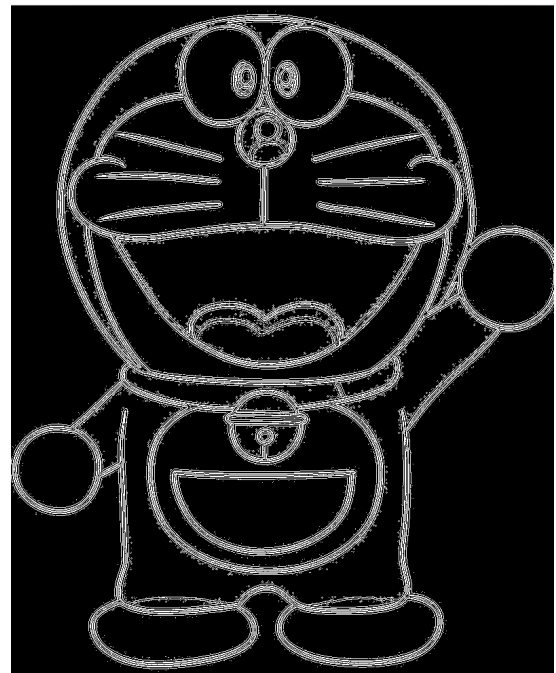


FIGURE 3.2 –

FIGURE 3.3 – Représentation de la fonction `convertScalesAbs`

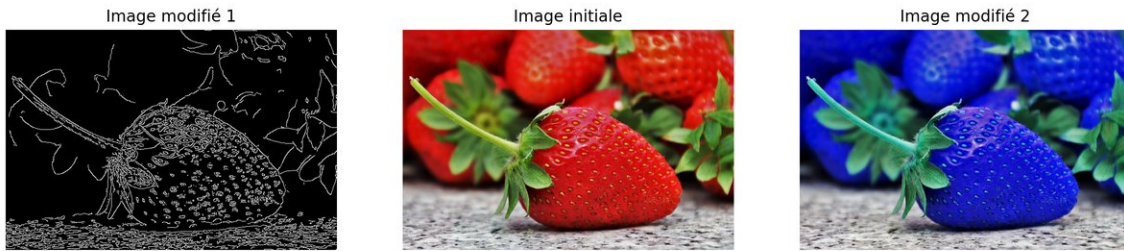
de la fonction. Il est possible de constater une homogénéisation des contours de la figure. Ceci est en accord avec sa définition, en effet après un recherche sur internet, on sait que la fonction met à l'échelle, calcule les valeurs absolues et convertit le résultat en 8 bits, ce qui la rend bien plus performante pour un traitement d'image que le changement de type `np.uint8`.

3.0.11 Question 5)

Dans cette question bonus, nous vous présentons nos plus belles modifications d'images sans passer par un logiciel de traitement de l'image.

Nous vous présentons tout d'abord sur la première modification le côté obscur de la fraise et sur la deuxième modification le côté clair de la force.

Pour ce faire nous avons utilisé la fonction Canny qui récupère uniquement les bords de l'image choisie et les met au premier rang en supprimant ceux inutiles. C'est exactement ce que la fonction $f(U)$ fait mais avec une autre méthode.



Ensuite nous avons croisé un lion qui était tombé au beau milieu du village des Schtroumpf. En effet pour obtenir l'image modifié nous avons seulement modifié ce que la fonction renvoie. On utilise à la place de `cv2.imshow` un `plt.imshow()` et on obtient alors cette inversion de couleurs.



Nous nous sommes également amusés à effectuer une inversion des couleurs pour le tableau du célèbre peintre Claude Monet par la même méthode que celle présentée pour le lion.

Image initiale



Image modifié



La nuit étoilé n'a pas échappé à l'inversion des couleurs elle aussi toujours avec la méthode du changement de `plt.imshow`.

Image initiale



Image modifié



3.0.12 3) Diffusion anisotrope

Comme précédemment nous allons nous intéresser à l'équation de diffusion modifiée :

$$(P_{aniso}) = \begin{cases} \frac{\partial u}{\partial t}(t, x, y) = \nabla \cdot (c(|\nabla u(t, x, y)|^2) \nabla u(t, x, y)) \\ u(0, x, y) = u_0(x, y) \\ \frac{\partial u}{\partial x}(t, x, 0) = \frac{\partial u}{\partial x}(t, x, 1) = 0 \\ \frac{\partial u}{\partial x}(t, 0, y) = \frac{\partial u}{\partial x}(t, 1, y) = 0 \end{cases}$$

où $\nabla(\bullet)$ note l'opérateur divergence et c l'une des fonctions suivantes :

$$c(s) := \frac{1}{1 + \frac{s^2}{\lambda^2}} \quad \text{et} \quad c(s) := e^{-\frac{s^2}{\lambda^2}}$$

avec $\lambda > 0$ fixé. Dans la partie programmation nous testerons l'influence du paramètre λ .

Comme vous pouvez le constater les formules présentées ci dessus pour $c(s)$ sont différentes de celles du sujet. En effet après réflexion, il ne nous semble pas pertinent d'utiliser le s au carré car cela signifie que la fonction elle même serait élevée au carrée. Or on sait que par exemple au dénominateur on doit avoir $2\Delta x$, si on utilisait la fonction c au carré on aurait $4\Delta x^2$ ce qui n'est pas correct.

3.0.13 Question 1)

Dans cette première question, nous posons le système suivant selon x :

$$\begin{cases} c_{i,j}^t := c\left(\left(\frac{u_{i+1,j}^t - u_{i-1,j}^t}{2\Delta x} + \frac{u_{i,j+1}^t - u_{i,j-1}^t}{2\Delta y}\right)\right) \\ \left(c \frac{\partial u}{\partial x}\right)_{i+\frac{1}{2},j} \approx \left(\frac{c_{i,j}^t + c_{i+1,j}^t}{2}\right) \left(\frac{u_{i+1,j}^t - u_{i,j}^t}{\Delta x}\right) := \phi_{i+\frac{1}{2},j} \\ \left(\frac{\partial}{\partial x} \left(c \frac{\partial u}{\partial x}\right)\right) \approx \frac{\phi_{i+\frac{1}{2},j} - \phi_{i-\frac{1}{2},j}}{\Delta x} \end{cases}$$

on pose également le système suivant selon y :

$$\begin{cases} c_{i,j}^t := c\left(\left(\frac{u_{i+1,j}^t - u_{i-1,j}^t}{2\Delta x} + \frac{u_{i,j+1}^t - u_{i,j-1}^t}{2\Delta y}\right)\right) \\ \left(c \frac{\partial u}{\partial y}\right)_{i,j+\frac{1}{2}} \approx \left(\frac{c_{i,j}^t + c_{i,j+1}^t}{2}\right) \left(\frac{u_{i,j+1}^t - u_{i,j}^t}{\Delta y}\right) := \phi_{i,j+\frac{1}{2}} \\ \left(\frac{\partial}{\partial y} \left(c \frac{\partial u}{\partial y}\right)\right) \approx \frac{\phi_{i,j+\frac{1}{2}} - \phi_{i,j-\frac{1}{2}}}{\Delta y} \end{cases}$$

On cherche à calculer les dérivées partielles de x et de y telles que :

$$\frac{du_{i,j}^t}{dt} \approx A_{x,i,j} + A_{y,i,j}$$

où

$$\begin{aligned} A_{x,i,j} &= \frac{1}{2\Delta x^2} [(c_{i,j} + c_{i-1,j})u_{i-1,j} - (c_{i-1,j} + 2c_{i,j} + c_{i+1,j})u_{i,j} + (c_{i,j} + c_{i+1,j})u_{i+1,j}] \\ A_{y,i,j} &= \frac{1}{2\Delta y^2} [(c_{i,j} + c_{i,j-1})u_{i,j-1} - (c_{i,j-1} + 2c_{i,j} + c_{i,j+1})u_{i,j} + (c_{i,j} + c_{i,j+1})u_{i,j+1}] \end{aligned}$$

Afin de retrouver l'expression donnée dans l'énoncé, on utilise l'équation de diffusion anisotrope. Puis on utilise la définition de la divergence en coordonnées cartésiennes telle que :

$$\begin{aligned}
 \frac{\partial}{\partial t} \left(\frac{\partial u}{\partial x} \right) &= \frac{\partial}{\partial x} \left(\frac{\partial u}{\partial t} \right) = \frac{\partial}{\partial x} \left(\nabla \cdot (c |\nabla u|^2 \nabla u) \right) \\
 &= \nabla \cdot \left(c |\nabla u|^2 \frac{\partial}{\partial x} \nabla u \right) + \frac{\partial c}{\partial x} |\nabla u|^2 \frac{\partial u}{\partial x} + 2c \frac{\partial}{\partial x} |\nabla u| \frac{\partial |\nabla u|}{\partial x} \frac{\partial u}{\partial x} \frac{\partial}{\partial t} \left(\frac{\partial u}{\partial y} \right) \\
 &= \frac{\partial}{\partial y} \left(\frac{\partial u}{\partial t} \right) = \frac{\partial}{\partial y} \left(\nabla \cdot (c |\nabla u|^2 \nabla u) \right) \\
 &= \nabla \cdot \left(c |\nabla u|^2 \frac{\partial}{\partial y} \nabla u \right) + \frac{\partial c}{\partial y} |\nabla u|^2 \frac{\partial u}{\partial y} + 2c \frac{\partial}{\partial y} |\nabla u| \frac{\partial |\nabla u|}{\partial y} \frac{\partial u}{\partial y} \nabla \cdot (c |\nabla u|^2 \nabla u) \\
 &= \frac{\partial}{\partial x} (c |\nabla u|^2 \frac{\partial u}{\partial x}) + \frac{\partial}{\partial y} (c |\nabla u|^2 \frac{\partial u}{\partial y})
 \end{aligned}$$

On a donc finalement :

$$\frac{du_{i,j}^t}{dt} = \frac{\partial}{\partial x} (c \frac{\partial u}{\partial x}) + \frac{\partial}{\partial y} (c \frac{\partial u}{\partial y})$$

Nous pouvons ainsi commencer par le terme en x, en rappelant que :

$$\frac{\partial}{\partial x} (c \frac{\partial u}{\partial x}) = \frac{\phi_{i+\frac{1}{2},j} - \phi_{i-\frac{1}{2},j}}{\Delta x}$$

Or on sait que : $\phi_{i+\frac{1}{2},j} = (\frac{c_{i,j}^t + c_{i+1,j}^t}{2}) (\frac{u_{i+1,j}^t - u_{i,j}^t}{\Delta x})$

Ainsi : $\phi_{i-\frac{1}{2},j} = (\frac{c_{i-1,j}^t + c_{i,j}^t}{2}) (\frac{u_{i,j}^t - u_{i-1,j}^t}{\Delta x})$

Dans cette partie, la fonction ne prend pas en compte le temps, ainsi on se permet d'enlever t.

On remplace dans l'équation de base et on trouve :

$$\begin{aligned}
 \frac{\partial}{\partial x} (c \frac{\partial u}{\partial x}) &\approx \frac{1}{\Delta x} \left[(\frac{c_{i,j} + c_{i+1,j}}{2}) (\frac{u_{i+1,j} - u_{i,j}}{\Delta x}) - (\frac{c_{i-1,j} + c_{i,j}}{2}) (\frac{u_{i,j} - u_{i-1,j}}{\Delta x}) \right] \\
 &\Leftrightarrow \approx \frac{1}{2\Delta x^2} [(c_{i,j} + c_{i+1,j})(u_{i+1,j}) - (c_{i,j} + c_{i+1,j})(u_{i,j}) - (c_{i-1,j} + c_{i,j})(u_{i,j}) + (c_{i-1,j} + c_{i,j})(u_{i-1,j})] \\
 &\Leftrightarrow \frac{\partial}{\partial x} (c \frac{\partial u}{\partial x}) \approx \frac{1}{2\Delta x^2} [(c_{i-1,j} + c_{i,j})(u_{i-1,j}) - (c_{i+1,j} + 2c_{i,j} + c_{i-1,j})(u_{i,j}) + (c_{i,j} + c_{i+1,j})(u_{i+1,j})]
 \end{aligned}$$

Ainsi :

$$\Leftrightarrow \frac{\partial}{\partial x} (c \frac{\partial u}{\partial x}) \approx A_{x,i,j}$$

On peut ensuite calculer pour le terme en y, en rappelant que :

$$\frac{\partial}{\partial y} (c \frac{\partial u}{\partial y}) = \frac{\phi_{i,j+\frac{1}{2}} - \phi_{i,j-\frac{1}{2}}}{\Delta y}$$

Or on sait que : $\phi_{i,j+\frac{1}{2}} = \left(\frac{c_{i,j}+c_{i,j+1}}{2}\right)\left(\frac{u_{i,j+1}-u_{i,j}}{\Delta y}\right)$

Ainsi : $\phi_{i,j-\frac{1}{2}} = \left(\frac{c_{i,j-1}+c_{i,j}}{2}\right)\left(\frac{u_{i,j}-u_{i,j-1}}{\Delta y}\right)$

On remplace ensuite dans l'équation de base et on trouve :

$$\begin{aligned} \frac{\partial}{\partial y}(c \frac{\partial u}{\partial y}) &\approx \frac{1}{\Delta y} \left[\left(\frac{c_{i,j}+c_{i,j+1}}{2}\right)\left(\frac{u_{i,j+1}-u_{i,j}}{\Delta y}\right) - \left(\frac{c_{i,j-1}+c_{i,j}}{2}\right)\left(\frac{u_{i,j}-u_{i,j-1}}{\Delta y}\right) \right] \\ \Leftrightarrow &\approx \frac{1}{2\Delta y^2} [(c_{i,j}+c_{i,j+1})(u_{i,j+1}) - (c_{i,j}+c_{i,j+1})(u_{i,j}) - (c_{i,j-1}+c_{i,j})(u_{i,j}) + (c_{i,j-1}+c_{i,j})(u_{i,j-1})] \\ \Leftrightarrow &\frac{\partial}{\partial y}(c \frac{\partial u}{\partial y}) \approx \frac{1}{2\Delta y^2} [(c_{i,j-1}+c_{i,j})(u_{i,j-1}) - (c_{i,j+1}+2c_{i,j}+c_{i,j-1})(u_{i,j}) + (c_{i,j}+c_{i,j+1})(u_{i,j+1})] \end{aligned}$$

Ainsi :

$$\Leftrightarrow \frac{\partial}{\partial y}(c \frac{\partial u}{\partial y}) \approx A_{y,i,j}$$

Finalement on a bien :

$$\frac{du_{i,j}^t}{dt} = \frac{\partial}{\partial x}(c \frac{\partial u}{\partial x}) + \frac{\partial}{\partial y}(c \frac{\partial u}{\partial y}) \approx A_{x,i,j} + A_{y,i,j}$$

3.0.14 Question 2)

Ici nous allons indenter en python les équations vu précédemment afin de mieux comprendre et voir le phénomène de diffusion anisotrope.

Programme de la diffusion anisotrope :

```

1  def c1(s,lam):
2      return (1/(1+(s**2/lam**2)))
3
4
5  def c2(s,lam):
6      return np.exp(-(s**2/lam**2))
7
8
9  def f_anisotrope(U):
10     m,n=np.shape(U)
11     C=np.zeros((m,n))
12     for i in range(1,m-1):
13         for j in range(1,n-1):
14             C[i,j]=c1((((U[i+1,j]-U[i-1,j])/2)**2+((U[i,j+1]-U[i,j-1])/2)**2)
15                 *(1/2),20)
16
17     Axi=np.zeros((m,n))
18     Ayi=np.zeros((m,n))
19     for i in range(1,m-1):
20         for j in range(1,n-1):
21             Axi[i,j] = (1/2) * (((C[i,j])+C[i-1,j])*U[i-1,j] - (C[i-1,j]+2*C[i,
22                 j]+C[i+1,j])*U[i,j] + (C[i,j]+C[i+1,j])*U[i+1,j])
23             Ayi[i,j] = (1/2) * ((C[i,j]+C[i,j-1])*U[i,j-1] - (C[i,j-1]+2*C[i,j]
24                 +C[i,j+1])*U[i,j] + (C[i,j]+C[i,j+1])*U[i,j+1])
25     return Axi + Ayi
26
27 def RK_anisotrope(U,t0,h,nbiter) :
28     U=U.astype(np.float64)
29     m,n,c=np.shape(U)

```

```

29 L=np.zeros((m+2,n+2,c))
30 for j in range(c):
31     I=completion(U[:, :, j])
32     for i in range(nbiter):
33         k1=h*f_anisotrope(I)
34         k2=h*f_anisotrope(I+k1/2)
35         k3=h*f_anisotrope(I+k2/2)
36         k4=h*f_anisotrope(I+k3)
37         I=I+(1/6)*(k1+2*k2+2*k3+k4)
38         print('iteration {}: {}'.format(i))
39
40     L[:, :, j]=I
41 return cv2.convertScaleAbs(L)

```

A l'aide des fonctions précédentes, nous avons réussi à medeliser la diffusion anisotrope avec les parametres suivant : $\lambda = 8$ / $h = 0,6$ et nbiter=8

Image initiale



Image modifié



Image initiale



Image modifié



Image initiale



Image modifié



Nous avons pu observer que la fonction $c1$ avait moins d'effert de "lissage" que la fonction $c2$. De plus, plus le pas h est petit, plus il faudra d'itération pour lisser notre image.

Afin de conclure ce rapport, nous souhaitons tout d'abord remercier nos professeurs de mathématiques M. EL MAHBOUBY et M. COUFFIGNAL pour le temps qu'ils ont consacré à nous aider dans la compréhension de ce BE. Ce bureau d'étude a été pour nous l'occasion de travailler en équipe. Nous avons fait face tous ensemble aux différentes difficultés rencontrées.

Une difficulté majeure que nous avons rencontrée a été de faire le lien entre la partie théorique et la modification d'images sur python. Il a été de même difficile d'interpréter et de donner un sens physique aux résultats trouvés, ce qui nous a demandé une grande quantité de temps, mais aussi d'une réflexion plus poussée que simplement trouver le résultat et le commenter.

Pour étayer notre propos nous avons parfois eu des problèmes de pixels incrustés dans nos images ce qui a entraîné de nombreuses modifications de notre code python, notamment la fonction RK4. En effet nous avons des sorties d'images avec plusieurs pixels qui dénaturaient la photo mais aussi, par exemple des problèmes pour appliquer le flou sur l'image de la route où il était quasi-inexistant.

Finalement, ce projet nous a permis de redécouvrir plus en profondeur le fonctionnement de l'algorithme Runge-Kutta 4 mais également celui de la fonction Verlet ainsi que leurs nombreuses propriétés et applications. Mais il nous a également permis de consolider nos savoirs sur le module lié à la résolution d'équations différentielles numériques.